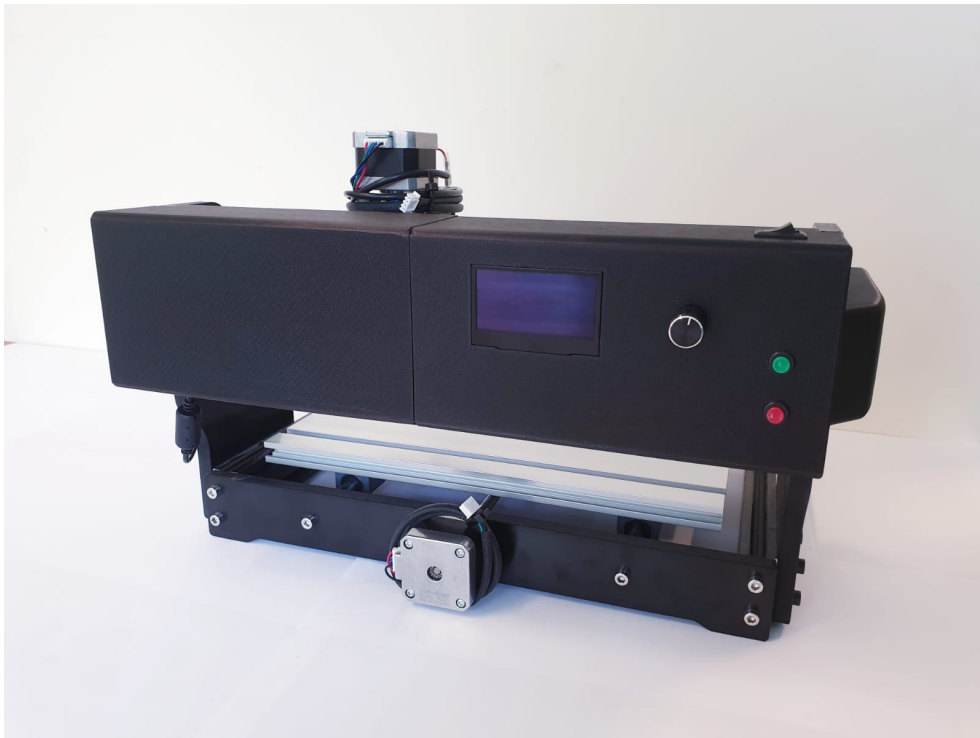




Chantal Crety, Jessica Perewersenko, Hannah Mey

Demonstrator Schrittmotor

Antrieb eines Schrittmotors mit dem Arduino Nano 33 BLE Sense

July 12, 2025

Contents

Contents	3
List of Figures	7
List of Listings	9
1. Projektbeschreibung	13
1.1. Einleitung	13
1.2. Zielsetzung	13
1.3. Umsetzung	13
1.4. Ergebnis	14
I. Arduino IDE	15
2. Arduino IDE 2.3.6	17
2.1. Beschreibung der Arduino IDE	17
2.2. Installation of the Arduino IDE	17
2.2.1. Download of the Arduino IDE	17
2.3. Übersicht	18
2.4. Funktionen	18
2.4.1. Sketchbook	19
2.4.2. Boards Manager	19
2.4.3. Library Manager	20
2.5. Beispiele	20
2.6. Debugging	21
3. Setup für den Arduino Nano 33 BLE Sense	23
3.1. Einführung	23
3.2. Konfiguration für den Arduino Nano 33 BLE Sense	23
3.2.1. Installation des Treibers	23
3.2.2. Verbindung und Konfiguration des Arduino-Boards mit dem Computer	24
3.2.3. Auswahl des passenden Ports	24
3.2.4. Sketch hochladen und überprüfen	26
3.3. Konfiguration testen	27
3.3.1. Test anhand des Beispiel-Sketches blink.ino	27
II. Arduino Nano 33 BLE Sense - Onboard Sensoren	29
4. Hardwarebeschreibung des Arduino	31
4.1. Aufbau des Arduinos	31
4.2. Integrierte Sensorik	32
4.2.1. 9-Achs-IMU für die Bewegungserkennung (LSM9DS1)	32
4.2.2. Näherungs-,Umgebungslicht-, Farb- und Gestensensor (APDS9960)	32
4.2.3. Barometrischer Drucksensor (LPS22HB)	33

4.2.4.	Sensor für relative Luftfeuchtigkeit und Temperatur (HTS221)	33
4.2.5.	Digitales Mikrophon (MP34DT05)	33
4.2.6.	DC-DC-Wandler (MPM3610)	33
4.3.	Beschreibung der Schnittstellen	34
4.3.1.	I ² C	34
4.3.2.	USB	35
4.3.3.	Bluetooth®5	35
4.3.4.	Weitere Kommunikationsschnittstellen	35
4.3.5.	Digitale Ein- und Ausgangspins	35
4.3.6.	Analoge Eingangspins	36
4.3.7.	Weitere Pins	36
4.3.8.	Reset-Knopf	36
4.3.9.	LED-Lampen	36
4.4.	Hinweis Arduino 33 BLE Sense Lite	37
4.5.	Bezugsquellen	37

III. Arduino Nano 33 BLE Sense - External Sensors and Actors 39

5.	Schrittmotor NEMA 17	41
5.1.	Beschreibung eines Schrittmotors	41
5.1.1.	Aufbau und Funktionsweise eines Schrittmotors	41
5.1.2.	Schrittmotor Bauformen	42
5.1.3.	Betriebsarten unipolar und bipolar	43
5.1.4.	Mikroschrittverfahren	43
5.2.	Schrittverluste verhindern	44
5.2.1.	Auswahl des Schrittmotors	44
5.2.2.	Betriebsart	44
5.2.3.	Externe Ereignisse	46
5.3.	Beschreibung des verwendeten Schrittmotors	47
5.4.	Spezifikation	47
6.	Schrittmotortreiber DRV8825	49
6.1.	Allgemeine Beschreibung eines Motortreibers	49
6.2.	Spezifische Beschreibung des Schrittmotortreibers DRV8825	49
6.2.1.	Anschluss des Treibers mit dem Arduino Nano 33 BLE Sense	49
6.3.	Spezifikationen	50
6.4.	Kalibrierung	50
6.5.	Einfaches Beispiel mit Code	50
7.	Geschwindigkeitssensor LM393	53
7.1.	Allgemeine Beschreibung des Sensors	53
7.2.	Spezifische Beschreibung des Sensors	53
7.2.1.	Funktionsweise der Lichtschranke	53
7.2.2.	Signalverarbeitung mit dem LM393 Komparator	54
7.2.3.	Anschluss des Sensors mit dem Arduino Nano 33 BLE Sense	54
7.3.	Spezifikationen	54
7.4.	Bibliothek	54
7.4.1.	Installation	54
7.4.2.	Code-Beispiel	55
7.5.	Kalibrierung	56
7.6.	Einfaches Beispiel mit Code	56
7.7.	Tests	57

8. OLED-Display	59
8.1. Allgemeine Beschreibung eines OLED-Displays	59
8.2. Spezifische Beschreibung OLED-Displays	59
8.3. Anschluss des Sensors mit dem Arduino Nano 33 BLE Sense	59
8.4. Spezifikationen	60
8.5. Bibliothek	60
8.5.1. Installation	60
8.5.2. Code-Beispiel	60
8.6. Einfaches Beispiel mit Code	60
9. Drehwinkel-Encoder	63
9.1. Encoder	63
9.2. Drehwinkel-Encoder	63
 IV. Beschreibung des Hauptprogramms	 65
10. Beschreibung des Hauptprogramms	67
 V. Anhang	 73
11. Materialliste	75
Literaturverzeichnis	79
Stichwortverzeichnis	82
Index	83

List of Figures

2.1. Menüleiste	17
2.2. Übersicht	18
2.3. Sketchbook	19
2.4. Board Manager	19
2.5. Library Manager	20
2.6. Beispiele	20
3.1. Arduino Mbed OS Nano Board Installation	24
3.2. Verbundenes Board auswählen - hier Arduino Nano 33 BLE Sense	24
3.3. Arduino Nano 33 BLE Sense Reset Button	25
3.4. grüne und orangene Builtin-LED	26
3.5. Zum Upload von Arduino Sketches verfügbaren Port auswählen	26
3.6. Upload des Sketches auf das Arduino Board	27
3.7. Auswählen des Ports	27
3.8. LED Beispiel-Sketch	28
3.9. LED-Built Example	28
4.1. Arduino Nano 33 BLE Sense (Vereinfachte Darstellung)	31
4.2. Arduino Nano Pinbelegung [Ard25]	34
5.1. Prinzipieller Aufbau Schrittmotor (2-phasig) [Hag21]	42
5.2. Start-Stopp Frequenz [Fau20]	44
5.3. Trapezförmiges Geschwindigkeitsprofil [Fau20]	45
7.1. Aufrufen der Bibliotheken in der Arduino IDE	55
7.2. Installation der TimerOne Bibliothek	55
7.3. Aufrufen der Beispiele für die TimerOne Bibliothek	56
8.1. Installation der U8g2 Bibliothek	61

List of Listings

9.1. Testprogramm für den Encoder	64
---	----

Acronyms

AAR Automatic Address Recognition

ADC Analog to Digital Converter

AES Advanced Encryption Standard

BLE Bluetooth Low Energy

CCM Counter with CBC-MAC

DMA Direkt Memory Access

ECB Electronic Codebook

EMI Electromagnetic Interference

FPU Floating Point Unit

I²C Inter-Integrated Circuit

IC Inter Circuit

IMU Inertial Measurement Unit

LED Light Emitting Diode

MUSB Mini Universal Serial Bus

NFC Near Field Communication

OLED Organic Light Emitting Diode

PDM Pulse Density Modulation

SCL Serial Clock

SDA Serial Data

SMD Surface-Mounted Device

SoC System on Chip

SPI Serial Peripheral Interface

UV Ultraviolettstrahlung

1. Projektbeschreibung

1.1. Einleitung

Im Rahmen dieses Projekts wurde ein vorhandener CNC-Tisch erweitert, um einen seiner Schrittmotoren als Demonstrator mit variabler Geschwindigkeitsregelung zu betreiben. Ziel war es, die technische Integration und Ansteuerung über einen Arduino Nano 33 BLE Sense umzusetzen und gleichzeitig eine benutzerfreundliche Bedienoberfläche zu schaffen.

1.2. Zielsetzung

Die Hauptaufgabe bestand darin, einen Schrittmotor des CNC-Tisches in mehreren Geschwindigkeitsstufen betreiben zu können. Die Steuerung sollte über einen Arduino Nano 33 BLE Sense erfolgen, wobei die Auswahl der Geschwindigkeit über Bedienelemente möglich sein sollte. Zusätzlich sollte ein optischer Geschwindigkeitssensor vom Typ LM393 die Drehzahl erfassen und zur Kontrolle auf einem Display ausgeben.

1.3. Umsetzung

Die Umsetzung des Projektes erfolgte in mehreren Phasen:

Elektronische Integration:

- Anschluss des A8825-Motortreibers an den Arduino
- Anbindung des LM393-Sensors zur Geschwindigkeitsmessung
- Testbetrieb auf einem separaten Testaufbau vor Montage am CNC-Tisch

Programmierung:

- Entwicklung eines Arduino-Codes zur Ansteuerung des Schrittmotors in verschiedenen Geschwindigkeitsstufen
- Implementierung der Auswertung des Sensorsignals zur Bestimmung der Drehzahl
- Bereitstellung einfacher Benutzerinteraktion (über Dreh-Encoder und OLED-Display)

Mechanische Montage:

- Befestigung der Bauteile direkt am CNC-Tisch
- Konstruktion und Montage einer Blende zur Aufnahme der Bedienelemente

Test und Inbetriebnahme:

- Durchführung von Funktionstests in allen Geschwindigkeitsstufen
- Abgleich der Sensordaten mit den programmierten Sollwerten
- Dokumentation der Ergebnisse

1.4. Ergebnis

Der Demonstrator zeigt erfolgreich den Betrieb eines CNC-Schrittmotors in mehreren konfigurierbaren Geschwindigkeitsstufen. Die Integration des Sensors erlaubt eine Live-Rückmeldung über die aktuelle Drehzahl. Die Bedienelemente sind über eine selbst konstruierte Blende zugänglich und ermöglichen eine einfache Steuerung durch den Nutzer.

Part I.

Arduino IDE

2. Arduino IDE 2.3.6

In diesem Kapitel werden die Funktionen und die Benutzeroberfläche der Arduino IDE erklärt. Die einzelnen Komponenten werden erklärt und es wird aufgezeigt, wie man diese benutzt. Des Weiteren gibt es eine Installationsanleitung und es wird aufgezeigt, wie der Arduino Nano 33 BLE Sense mit der Arduino IDE verknüpft wird.

2.1. Beschreibung der Arduino IDE

Die Arduino IDE ist eine open-source Software zur Bearbeitung, Kompilierung und Übertragung von Codes auf Arduino-Module. Sie ist plattformübergreifend nutzbar, basiert auf Java und ist mit Windows, macOS und Linux kompatibel. Die Arduino IDE unterstützt verschiedene Arduino-Module, sowie die Programmierung in C und C++.

Die Programme, welche in der Arduino IDE geschrieben werden nennt man "Sketches" und können auf den Mikrocontroller übertragen werden.

Wenn man die Arduino IDE öffnet, dann sieht man in der oberen Menüleiste Figure 2.1 5 Schaltflächen:



Figure 2.1.: Menüleiste

Die Schaltflächen haben folgende Funktionen (von links nach rechts):

- Verify: der Haken überprüft den Code auf Fehler und zeigt, ob er korrekt strukturiert ist.
- Upload: der Pfeil kompiliert den Code und überträgt ihn an das angeschlossene Board.
- Start Debugging: startet eine Debugging-Sitzung, um den Code Schritt für Schritt durchzugehen.
- Serial Plotter: Echtzeit-Daten vom Board werden grafisch angezeigt.
- Serial Monitor: Terminalfenster zum Senden und Empfangen von Textdaten in Echtzeit über die serielle Schnittstelle.

2.2. Installation of the Arduino IDE

2.2.1. Download of the Arduino IDE

Der Arduino Nano 33 BLE Sense verwendet die Arduino Softwareumgebung **Integrated Development Environment (IDE)** zur Programmierung. Diese IDE ist die am weitesten verbreitete für alle Arduino-Boards und kann sowohl online als auch offline verwendet werden. Die Arduino IDE ist eine open-source Software und vereinfacht das Schreiben von Code und das Hochladen auf das Board erheblich. Die Software ist für Windows, Linus und macOS verfügbar und die aktuellste Version kann von der offiziellen Arduino-Website heruntergeladen werden: [Arduino-Software](#)

2.3. Übersicht

Die Arduino IDE 2 verfügt über eine Seitenleise, die den einfachen Zugriff auf die am häufigsten verwendeten Werkzeuge ermöglicht. 2.2 [Ard24]

- **Verify / Upload:** Diese Funktionen dienen zum Kompilieren und Hochladen von Code auf ein Arduino-Board.
- **Select Board and Port:** Erkannte Arduino-Boards werden automatisch angezeigt, inklusive der zugewiesenen Portnummer.
- **Sketchbook:** Das Sketchbook ist ein zentraler Ort zur Verwaltung aller lokal gespeicherten Sketches. Es bietet eine übersichtliche Struktur zum Organisieren und Bearbeiten von Projekten und eine Synchronisierungsfunktion mit der Arduino Cloud, wodurch Sketches geräteübergreifend verfügbar sind.
- **Boards Manager:** Hier können offizielle Boardpakete von Drittanbietern installiert werden.
- **Library Manager:** Mit dem Bibliotheksverwalter hat man Zugriff auf tausente Arduino-Bibliotheken, welche von Arduino selbst oder der Community erstellt wurden.
- **Debugger:** Der Debugger ermöglicht das Testen und Debuggen von Programmen in Echtzeit.
- **Search:** Hier kann der geschriebene Coder auf Schlüsselwörter durchsucht werden.
- **Open Serial Monitor:** Hier wird der serielle Monitor als neuer Tab im unteren Konsolenbereich geöffnet.



Figure 2.2.: Übersicht

2.4. Funktionen

Die Hauptfunktionen der Arduino IDE sind die direkte Installation von Bibliotheken, die Synchronisierung von Sketches über die Cloud und die integrierten Debugging-Tools. Im Folgenden werden einige der zentralen Funktionen näher beschrieben.

2.4.1. Sketchbook

Das Sketchbook 2.3 ist der Speicherort der Arduino-Code-Dateien, die als Sketches bezeichnet werden. Diese Dateien werden mit der Endung ".ino" gespeichert. Dabei ist wichtig, dass jeder Sketch in einem Ordner gespeichert werden muss, der genau denselben Namen hat wie die Sketch-Datei. Als Beispiel: Der Sketch heißt "MySketch.ino", also muss der Ordner, in welchem sich der Sketch befindet, "MySketch" benannt werden.

Standardmäßig werden Sketches in einem Ordner namens Arduino gespeichert, der sich im Dokumenten-Verzeichnis des Betriebssystems befindet. Im linken Seitenmenü der Arduino IDE befindet sich ein Ordner-Symbol, über das man direkt zum Sketchbook gelangt. Dort können Sketches geöffnet, bearbeitet und verwaltet werden.



Figure 2.3.: Sketchbook

2.4.2. Boards Manager

Der Boards Manager ist über folgenden Pfad in der IDE erreichbar: **Tools** » **Board** » **Board Manager**.

In der Arduino IDE ermöglicht der Boards Manager die Installation verschiedener Board-Pakete, wie in Abbildung 2.4 gezeigt. Ein Board-Paket enthält alle notwendigen Dateien und Anweisungen, um Code für bestimmte Arduino-Boards zu kompilieren und hochzuladen.

Es sind verschiedene Board-Pakete verfügbar. Jedes Paket unterstützt verschiedene Arduino Boards, der Boards Manager stellt also sicher, dass die richtigen Tools und Compiler installiert sind und die Programmierung für das gewählte Board reibungslos funktioniert.

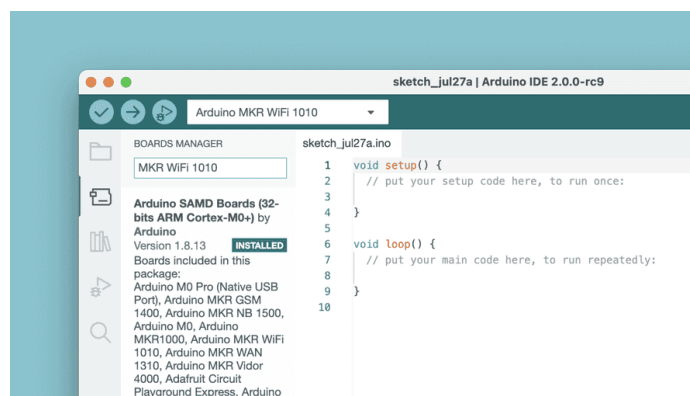


Figure 2.4.: Board Manager

2.4.3. Library Manager

Der Library Manager ist über folgenden Pfad in der IDE erreichbar: **Tools** » **Manage Libraries**. In der Arduino IDE ermöglicht der Library Manager das Durchsuchen und Installieren einer Vielzahl von Bibliotheken. Bibliotheken erweitern die grundlegenden Arduino-Funktionen und erleichtern komplexe Aufgaben wie das Auslesen spezieller Sensoren oder die Ansteuerung von Modulen. Diese Bibliotheken bieten vorgefertigten Code, eine Vereinfachung der Programmierung, die Unterstützung spezifischer Hardwarekomponenten und eine schnellere und effizientere Entwicklung. Der Library Manager ist in Abbildung 2.5 zu sehen.



Figure 2.5.: Library Manager

2.5. Beispiele

Ein zentraler Bestandteil der Arduino-Dokumentation ist die Vielzahl an Beispielsketches, die mit unterschiedlichen Bibliotheken mitgeliefert werden. Diese Beispiele demonstrieren die praktische Anwendung von Bibliotheksfunktionen und zeigen deren beabsichtigte Verwendung und deren Hauptfunktionen im Einsatz. Die Beispielsketches können aus vorinstallierten Bibliotheken stammen oder aus den Board-spezifischen Bibliothekspaketen. Zugriff auf die Beispiele erhält man über **File** » **Examples**, und dann in der List die gewünscht Bibliothek auswählen.

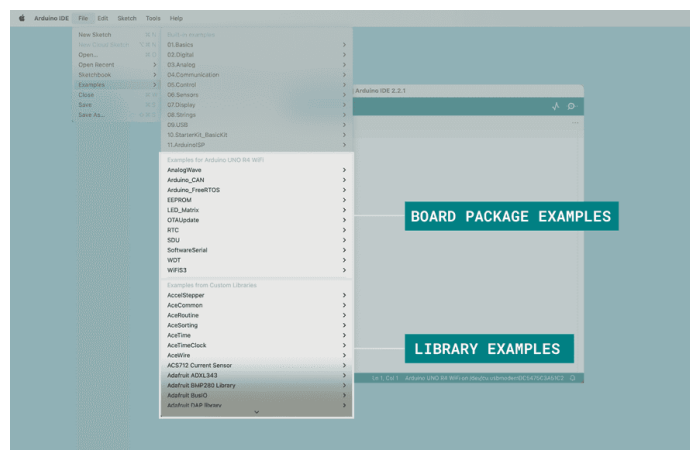


Figure 2.6.: Beispiele

In Abbildung 2.6 sieht man die Beispielliste, wenn ein Arduino UNO R4 WiFi an den Computer angeschlossen ist.

2.6. Debugging

Das Debugger Tool in der Arduino IDE dient dazu, Programme gezielt zu testen und zu debuggen. Zum Einen kann man mit dem Debugger Breakpoints setzen, um den Programmablauf an bestimmten Stellen zu unterbrechen. Zum Anderen gibt es die Funktion des schrittweisen Durchlaufend, um den Code Zeile für Zeile abzuarbeiten. Außerdem können Variablen überwacht werden, somit kann man Variablenwerte in Echtzeit einsehen, um Fehler besser einzugrenzen. Das Debugging bietet Vorteile, wie das Auffinden von Logikfehlern, das Überprüfen von Zustandsänderungen und es bietet Einsicht in den Speicherzustand während der Programmausführung.

3. Setup für den Arduino Nano 33 BLE Sense

3.1. Einführung

In diesem Kapitel wird der Prozess beschrieben, wie der Arduino Nano 33 BLE Sense mit einem Computer verbunden und die notwendigen Einstellungen vorgenommen werden, um mit der Programmierung zu beginnen.

Behandelt werden:

- die Installation erforderlicher Tools
- die die Konfiguration in der Arduino IDE
- ein einfacher Beispiel-Sketch zur Überprüfung der Funktion

Zur Überprüfung der Konfiguration kann einer der vordefinierten Beispiel-Sketches genutzt werden, z.B. der einfache `blnik.ino`, der mit der IDE ausgeliefert wird.

3.2. Konfiguration für den Arduino Nano 33 BLE Sense

Um den **Arduino Nano 33 BLE Sense** in einer offline Umgebung zu programmieren sind folgende Schritte notwendig:

3.2.1. Installation des Treibers

1. **Arduino IDE installieren:** Installieren Sie die aktuelle Version der Arduino IDE auf Ihrem Computer.
2. **Installation der Boardpakete:** Öffnen Sie die `Arduino IDE` und wählen Sie im Menü `Tools`, in der oberen linken Ecke.
3. Im Menü `Tools`, wählen Sie den `Board Manager`.
4. Suchen Sie im Fenster `Board Manager` nach **Arduino Nano 33 BLE Sense**.
5. Wählen Sie in den Suchergebnissen das Paket `Arduino Mbed OS Nano Boards` und klicken Sie `Install`, um das Paket zu installieren.
6. Schließen Sie das Board über einen USB-A zu Micro-USB Kabel an den Computer an.

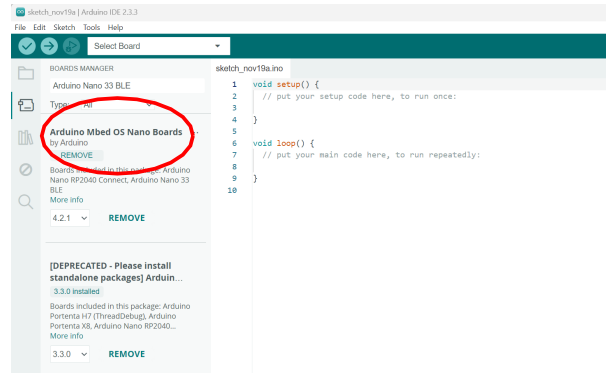


Figure 3.1.: Arduino Mbed OS Nano Board Installation

3.2.2. Verbindung und Konfiguration des Arduino-Boards mit dem Computer

Um entweder ein vordefiniertes Beispiel oder ein eigenes Programm auf dem Arduino Nano 33 BLE Sense auszuführen, befolgen Sie die folgenden Schritte::

1. Verbinden Sie das Arduino-Board mit dem Computer.
2. Öffnen Sie die Arduino IDE, dann erscheint ein leeres Projekt mit den Standardfunktionen `void setup()` und `void loop()`.
3. Gehen Sie im Menü zu **Tools**, wählen Sie **Board** und wählen Sie das angeschlossene Board **Arduino Nano 33 BLE Sense**, wie in Abbildung 3.2.

Damit ist die IDE so konfiguriert, dass sie das Board erkennt und mit ihm kommunizieren kann.

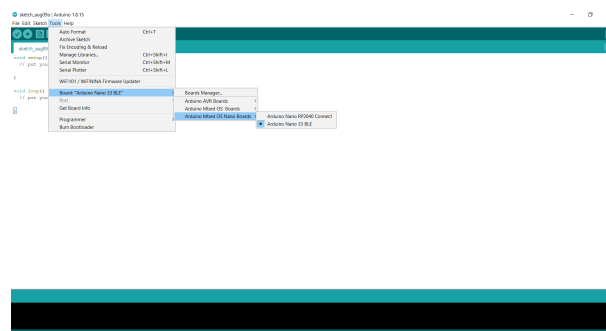


Figure 3.2.: Verbundenes Board auswählen - hier Arduino Nano 33 BLE Sense

3.2.3. Auswahl des passenden Ports

Nach der Auswahl des Arduino Nano 33 BLE Sense, muss nun der richtige Port ausgewählt werden. Dafür versetzen Sie das Board in den Bootloader-Modus, drücken Sie dazu den weißen Reset-Knopf (siehe Abbildung 3.4).

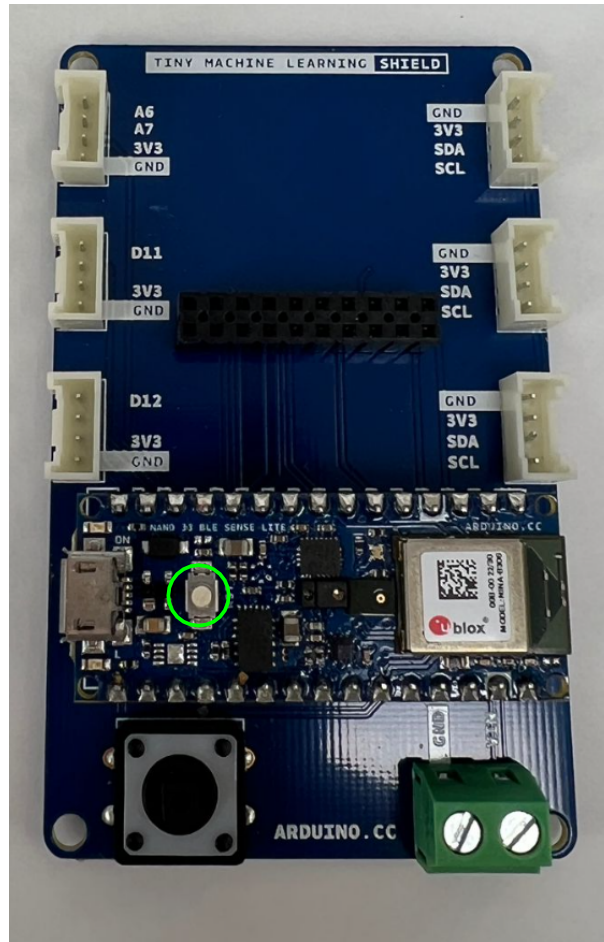


Figure 3.3.: Arduino Nano 33 BLE Sense Reset Button

Wenn der Bootloader-Modus aktiviert ist, leuchtet die orangefarbene LED, wie in Abbildung 3.4

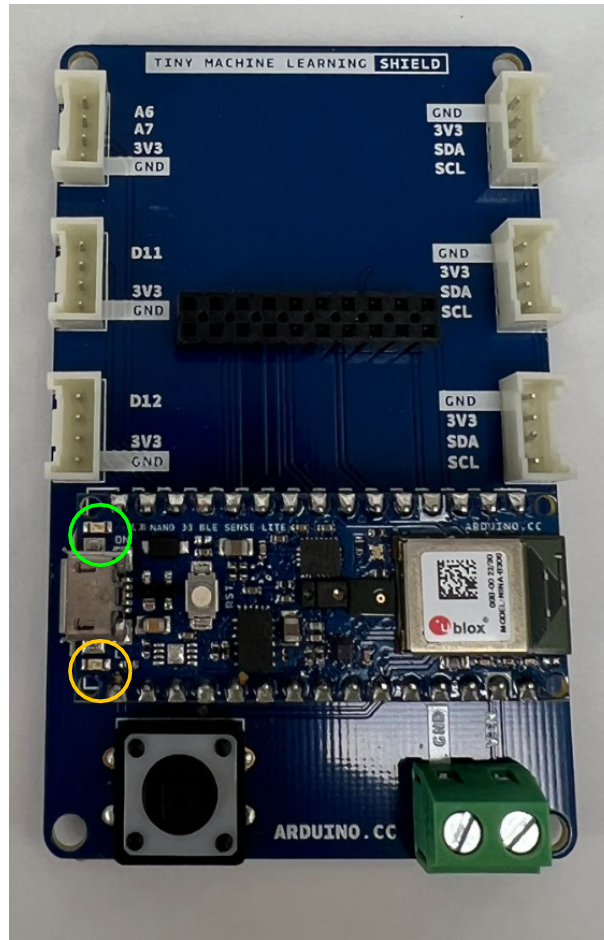


Figure 3.4.: grüne und orangene Builtin-LED

Nun müssen Sie den Port auswählen. Gehen Sie dazu zu **Tools**, wählen Sie **Port** und versichern Sie sich, dass der richtig Port ausgewählt ist. 3.5

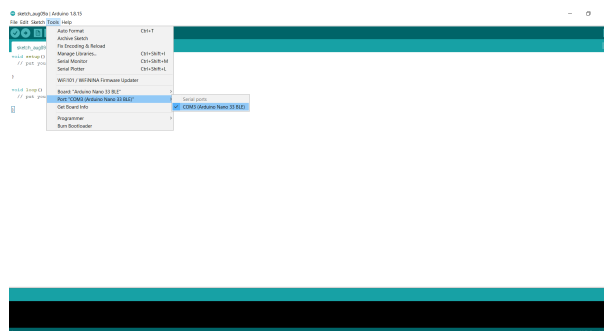


Figure 3.5.: Zum Upload von Arduino Sketches verfügbaren Port auswählen

3.2.4. Sketch hochladen und überprüfen

Nachdem der passende Port ausgewählt wurde, folgt nun der nächste Schritt: das Hochladen eines Sketches. Vor dem Hochladen sollte der Sketch mit der Schaltfläche "Verify" kompiliert werden, dadurch wird geprüft, ob der Code fehlerfrei ist. Nach

erfolgreicher Überprüfung kann der Sketch mit der Schaltfläche "Upload" an das Arduino Board übertragen werden. (Abbildung 3.6)

Als nächstes muss die Verbindung bestätigt werden. Unten rechts im Fenster der Arduino IDE sollte der Name des angeschlossenen Boards und der verwendete Port angezeigt werden, das bestätigt, dass das Board korrekt erkannt wurde.

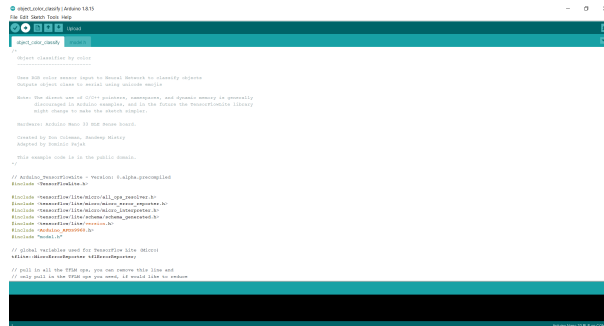


Figure 3.6.: Upload des Sketches auf das Arduino Board

Nach dem Hochladen wird der Sketch kompiliert. Falls dabei Fehler auftreten, werden sie im unteren schwarzen Konsolenbereich der Arduino IDE angezeigt. Nach dem erfolgreichen Upload sollte der verwendete Port erneut ausgewählt werden, um z.B. die Ausgabe im seriellen Monitor anzeigen zu können.

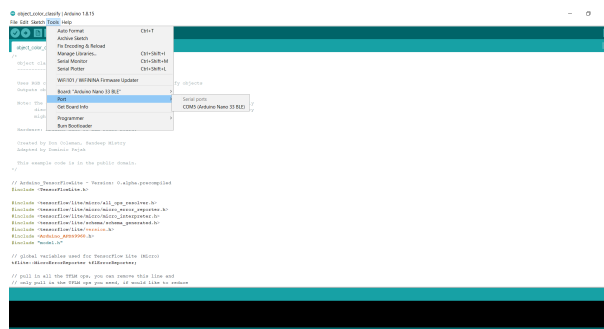


Figure 3.7.: Auswählen des Ports

3.3. Konfiguration testen

In diesem Abschnitt wird überprüft, ob das Board korrekt mit der Arduino IDE verbunden ist. Dazu wird ein einfacher Beispiel-Sketch ("Blink") ausgeführt, bei dem eine eingebaute LED blinkt. So lässt sich sowohl die Verbindung als auch die Grundfunktionalität des Boards überprüfen.

3.3.1. Test anhand des Beispiel-Sketches `blink.ino`

Die Arduino-IDE enthält eine Reihe integrierter Beispiele zu Testzwecken. Um die Konfiguration zu überprüfen und das Board einzurichten, kann zunächst das einfache Beispiel `blink.ino` geöffnet werden. Nach dem Hochladen des Beispiels `blink.ino` blinkt die eingebaute LED mit 2 Hz.

Dafür werden folgende Schritte ausgeführt:

1. **Beispiele öffnen:** In der Arduino IDE auf `File` `Examples` klicken.

2. **Sketch auswählen:** Eine Liste von Beispiel-Sketches wird angezeigt. Im Untermenü **01.Basics** **Blink** auswählen, wie in Abbildung 3.8 gezeigt.
3. **Blink Beispiel-Sketch:** Der Beispiel-Sketch `blink.ino` erscheint in einem neuen Fenster der Arduino IDE 3.9.
4. **Sketch hochladen:** Mit einem Klick auf den **Upload** Button wird der Sketch hochgeladen. Wenn der Upload erfolgreich ist, dann beginnt die eingebaute LED mit einer Frequenz von 2 Hz zu blinken.

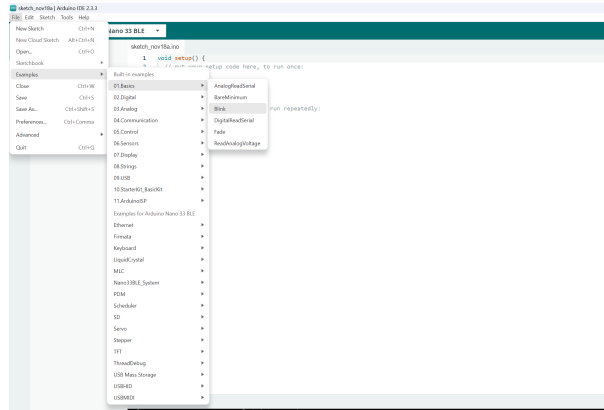


Figure 3.8.: LED Beispiel-Sketch

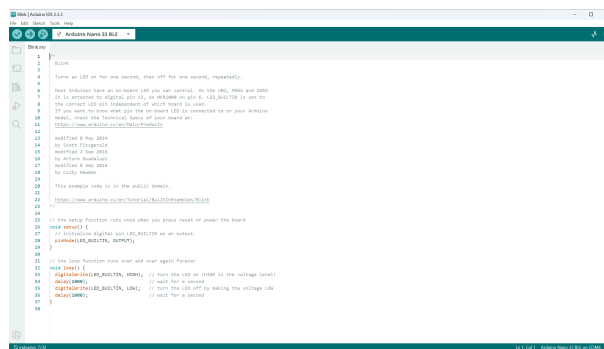


Figure 3.9.: LED-Built Example

Wenn der Test erfolgreich ist, dann ist somit bestätigt, dass die Verbindung zur Arduino IDE funktioniert und die Hardware betriebsbereit ist.

Part II.

**Arduino Nano 33 BLE Sense -
Onboard Sensoren**

4. Hardwarebeschreibung des Arduino

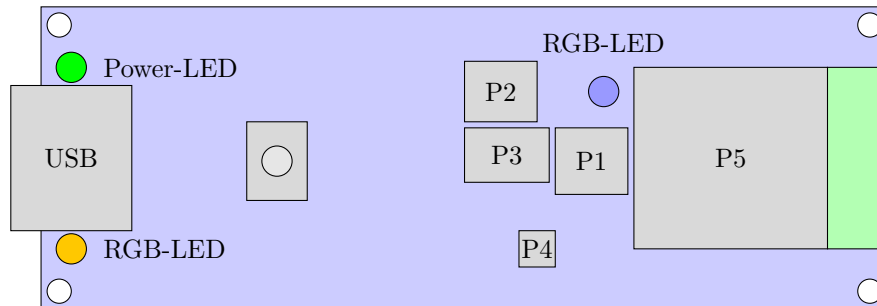


Figure 4.1.: Arduino Nano 33 BLE Sense (Vereinfachte Darstellung)

Pos.Nr.	Modul/Sensor
P1	Mikrophone
P2	9-Achs-IMU
P3	Näherungs-,Umgebungslicht-, Farb- und Gestensensor
P4	Drucksensor
P5	Bluetooth-Modul

Table 4.1.: Arduino Nano 33 BLE Sense [Ard25]

4.1. Aufbau des Arduinos

Der Arduino Nano 33 Bluetooth Low Energy (BLE) Sense ist ein Mikrocontroller basierend auf dem Nordic nRF52840-System on Chip (SoC). Trotz seiner kompakten Bauweise mit einer Größe von 45 x 18 mm verfügt es über mehrere verschiedene integrierte Sensoren, Aktoren und Anschlussschnittstellen (siehe Figure 4.1). Darüber hinaus verfügt der Arduino-Prozessor über 1 MB Flash-Speicher und 256 KB RAM. Bei dem Prozessor handelt es sich um einen Arm®Cortex-M4F Prozessorkern, welcher mit einer Taktrate von 64 MHz arbeitet und für Mikrocontroller Anwendungen optimiert ist. Dieser bietet eine gute Leistung bei geringen Stromverbrauch und ist auch für Echtzeit-Verarbeitungsaufgaben geeignet. Außerdem besitzt der Prozessor eine Floating Point Unit (FPU), die eine effiziente Abarbeitung von Datenverarbeitungsoperationen ermöglicht. Mit Hilfe einer integrierten Steckverbindung (engl. „header“) kann der Mikrocontroller direkt auf ein Board gesteckt werden. Der Name Arduino Nano BLE 33 Sense gibt bereits Aufschluss über einiger Eigenschaften und Funktionen des Mikrocontrollers:

- "Nano": steht für die kleine Bauform
- "33": bedeutet, dass das Board mit 3,3 Volt betrieben wird
- "BLE": zeigt die Unterstützung für Bluetooth Low Energy
- "Sense": weist auf die integrierten Sensoren hin.

[Arm20] [Ard25]

4.2. Integrierte Sensorik

Der Arduino Nano 33 BLE-Sense verfügt über eine Vielzahl von integrierten Sensoren und Aktoren, mit denen verschiedene Umgebungseigenschaften erfasst werden können. Dazu gehören die in den folgenden Abschnitten beschriebenen Sensoren und Aktoren.

4.2.1. 9-Achs-IMU für die Bewegungserkennung (LSM9DS1)

Bei der Inertialmesseinheit LSM9DS1 handelt es sich um ein System-in-Package, d.h. es sind mehrere elektronische Komponenten bzw. Chips auf kleinstem Raum in einem Gehäuse kombiniert.[Lienig.2012] Die Inertial Measurement Unit (IMU) verfügt über einen 3D-Linearbeschleunigungsmesser, ein 3D-Gyroskop und einen 3D-Magnetometer. Darüber hinaus verfügt das System über eine serielle Inter-Integrated Circuit (I²C)-Busschnittstelle, die einen Standard und einen Fast Mode (100/400 kHz) bereitstellt, zudem eine serielle SPI-Standardschnittstelle. Der Sensor verfügt über einen linearen Beschleunigungsmesser (Accelerometer) mit wählbarer Skala von $\pm 2/\pm 4/\pm 8/\pm 16$ g, der die lineare Beschleunigung in drei Achsen (x, y, z) misst und die Erfassung von Geschwindigkeits- und Positionsänderungen ermöglicht. Das Magnetfeld ist mit einer wählbaren Skala von $\pm 4/\pm 8/\pm 12/\pm 16$ Gs ausgestattet, damit misst es das magnetische Feld in den drei Achsen und ermöglicht die Bestimmung der Ausrichtung relativ zur Erdmagnetfeldrichtung. Das 3D-Gyroskop ist mit einem wählbarem Skalenendwert von: $\pm 125/\pm 250/\pm 500/\pm 1000/\pm 2000 \frac{\text{Grad}}{\text{s}}$ ausgestattet und ist für die Erfassung der Winkelgeschwindigkeit bzw. Drehbewegung um die drei Achsen zuständig.[STM15][Ard25] Mögliche Anwendungsbereiche sind zum Beispiel eine Bewegungssteuerungen (Drohnensteuerung, Robotik und industrielle Automatisierung), Schwingungsüberwachung und -kompensation, Antennen, Plattformen, optische Bild- und Objektivstabilisierung.

4.2.2. Näherungs-,Umgebungslicht-, Farb- und Gestensensor (APDS9960)

Hierbei handelt sich um einen sehr vielseitigen Sensor. Er dient zur Gestenerkennung, Farberkennung, Abstandsmessung und Umgebungslichtmessung. Die Gestenerkennung nutzt vier gerichtete Fotodioden, um reflektierte Infrarot-Energie (von der integrierten Light Emitting Diode (LED)) zu erfassen, um physikalische Bewegungsinformationen (d.h. Geschwindigkeit, Richtung und Entfernung) in digitale Informationen zu übersetzen. Die Näherungserkennung ermöglicht die Messung der Entfernung durch die Erkennung der reflektierten Infrarot-Energie, (von der integrierten LED) mit Hilfe einer Fotodiode. Erkennungsereignisse sind interruptgesteuert und treten immer dann auf, wenn das Näherungsergebnis die oberen und/oder unteren Schwellenwerte überschreitet. Der Sensor verfügt außerdem über Offset-Einstellregister zur Kompensation des System-Offsets, der durch unerwünschte Infrarot-Energier reflexionen am Sensor verursacht wird. Die Intensität der Infrarot-LED wird werkseitig eingestellt, so dass eine Kalibrierung der Endgeräte aufgrund von Bauteilschwankungen nicht erforderlich ist. Die Farb- und Umgebungslichterkennung liefert Daten zur roten, grünen, blauen Farbspektrum und Daten zum klaren Lichtintensität. Jeder der Kanäle R, G, B, C verfügt über einen Ultraviolettstrahlung (UV)- und Infrarot-Sperrfilter und einen speziellen Datenkonverter, der gleichzeitig 16-Bit-Daten erzeugt. Diese Architektur ermöglicht Anwendungen eine genaue Messung des Umgebungslichts zu messen und die Farbe zu erkennen, was es den Geräten wiederum ermöglicht, die Farbtemperatur zu berechnen und die Hintergrundbeleuchtung des Displays dementsprechend anzupassen.[Ava15][Ard25]

4.2.3. Barometrischer Drucksensor (LPS22HB)

Der LPS22HB ist ein sehr kompakter Absolutdrucksensor, der auf dem piezoresistiven Prinzip basiert. Er arbeitet als Barometer und verfügt über einen digitalen Ausgang. Zudem ist in diesem Drucksensor ein Temperatursensor integriert, mit welchem Druckmessungen zusätzlich kompensiert werden können. Das Gerät besteht aus einem Sensorelement und einer Inter Circuit (IC)-Schnittstelle, die eine Kommunikation zwischen der Sensoreinheit und der Anwendung über I²C oder Serial Peripheral Interface (SPI) ermöglicht. Das Sensorelement erfasst den absoluten Druck und besteht aus einer speziell gefertigten, hängenden Membran. Der LPS22HB ist in einem vollständig vergossenem Land-Grid-Array-Gehäuse untergebracht, das kleine Löcher aufweist. Durch diese Öffnung gelangt der externe Druck auf das Sensorelement. Der Betrieb des Sensors ist über einen Temperaturbereich von -40 °C bis +85 °C gewährleistet. [STM17][Ard25] Anwendungsbereiche für diesen Sensor sind zum Beispiel: Wetterstationen, Höhenmesser, Luftdrucküberwachung in industriellen Prozessen oder tragbare smarte Geräte, wie Sportuhren oder Smartphones.

4.2.4. Sensor für relative Luftfeuchtigkeit und Temperatur (HTS221)

Der HTS221 ist ein kompakter Sensor zur Messung von relativer Luftfeuchtigkeit und Temperatur. Er enthält ein Sensorelement und eine Mischsignalverarbeitungseinheit, welche die gemessenen Daten über digitale Schnittstellen bereitstellt.

Der Sensor kann über I²C und auch über SPI-Bus angesprochen werden und ist für einen Temperaturbereich von -40°C bis +120°C geeignet. Die Temperaturmessgenauigkeit liegt bei dem HTS221 bei $\pm 0,5^{\circ}\text{C}$.

Anwendung findet dieser Sensor in Klimaanlage, Heiz- und Belüftungssystemen, Kühlschränken, Luftbefeuchtern oder in der industriellen Automatisierung. [Ard25]

4.2.5. Digitales Mikrofon (MP34DT05)

Das MP34DT05-A ist ein kompaktes, stromsparendes, omnidirektionales, digitales Mikrofon mit einem kapazitiven Sensorelement und einer IC-Schnittstelle. Das Sensorelement, das in der Lage ist, akustische Wellen zu detektieren, wird mit einem speziellen Silizium-Mikrobearbeitungsverfahren für die Herstellung von Audiosensoren hergestellt. Die IC-Schnittstelle wird in einem speziellen Halbleiterherstellungsverfahren (CMOS-Verfahren[GW22][Ber20]) hergestellt, das die Entwicklung einer speziellen Schaltung ermöglicht, die ein digitales Signal im Pulse Density Modulation (PDM)-Format extern bereitstellen kann. Das Mikrofon hat einen Signal-Rausch-Abstand von 64 dB und eine Empfindlichkeit von -26 dBFS ± 3 dB. Sein maximaler Schalldruckpegel liegt bei 122,5 dB SPL. Der MP34DT05-A ist in einem Surface-Mounted Device (SMD)-kompatiblen, Electromagnetic Interference (EMI)-geschirmten Top-Port-Gehäuse erhältlich und arbeitet zuverlässig über einen erweiterten Temperaturbereich von -40 °C bis +85 °C. Abmessungen des Gehäuses HCLGA-4 LD sind $3 \times 4 \times 1$ (in mm). Anwendung findet das Modul beispielsweise in mobilen Endgeräten, im Bereich der Spracherkennung sowie in Laptops und Notebooks.[STM21][Ard25]

4.2.6. DC-DC-Wandler (MPM3610)

Das MPM3610-Modul ist ein integriert Gleichspannungs- zu Gleichspannungs-Wandler. Dieser kann Eingangsspannungen von bis zu 21 V verarbeiten mit einem Wirkungsgrad von mindestens 65% bei minimaler Last. Bei einer Eingangsspannung erhöht sich der Wirkungsgrad auf 85%.[Ard25]

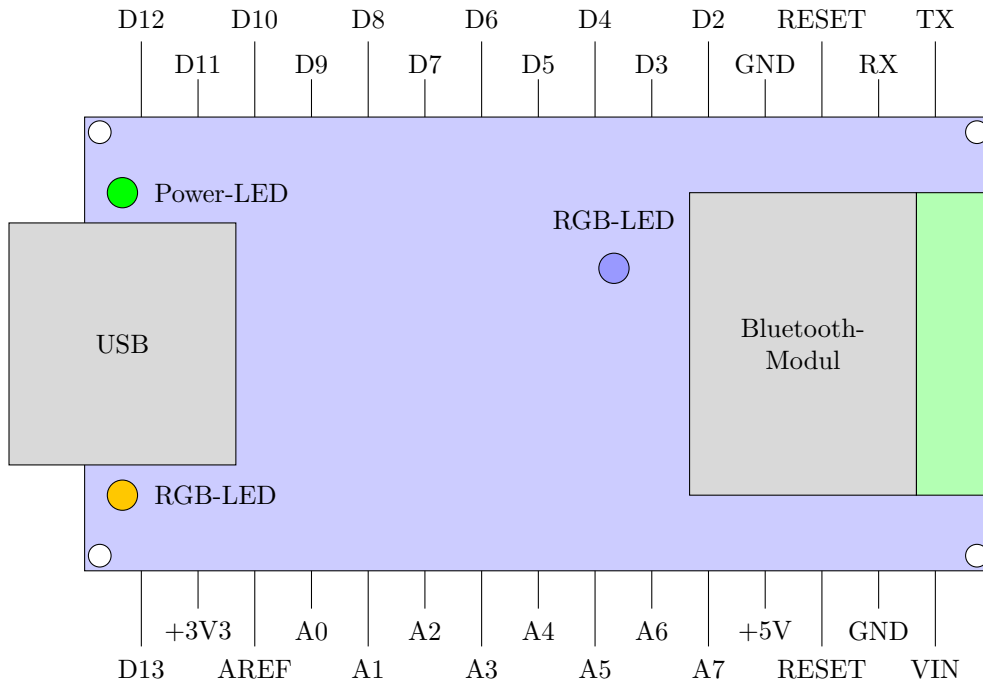


Figure 4.2.: Arduino Nano Pinbelegung [Ard25]

4.3. Beschreibung der Schnittstellen

Der Arduino Nano 33 BLE-Sense Lite bietet eine Vielzahl von Schnittstellen (siehe Figure 4.2), die das Board mit anderen Komponenten und Geräten einfach verbinden lassen. In diesem Abschnitt sind einige Details zu den wichtigsten Schnittstellen des Boards erläutert.

4.3.1. I²C

Der I²C-Bus ermöglicht die geräteinterne Kommunikation von Mikrocontrollern mit Sensoren, externen Bildschirmen, Echtzeituhren und vielen anderen Bausteinen. Das Kommunikationsprotokoll basiert auf dem Master-Slave-Prinzip. Der Master initiiert und beendet die Kommunikation, stellt den Takt zur Synchronisation bereit und löst Kommunikationsprobleme. Jeder Slave hat eine eindeutige Adresse, mit einer Länge von 7 oder 10 Bit und ermöglicht die Adressierung von bis zu 128 bzw. 1024 unterschiedlichen Slaves in dem selben Bus. Geräte mit unterschiedlichen Adresslängen können im selben Bus koexistieren. Bevor der Master Daten überträgt oder empfängt, muss er einen Slave mit einer vorher vereinbarten Adresse ansprechen.[MS21][STM15][Ber20] Sind ihre Adressen in Übereinstimmung kann im nächsten Schritt ein einzelnes Bit gesendet werden, mit dem dann festgelegt wird, ob der Master Daten an dem Slave übertragen (Binär:1) oder auslesen möchte (Binär:0). Dieser Datenaustausch wird danach bestätigt, sodass weitere Daten ausgetauscht werden können. Zur Beendigung des Datenaustausches wird ein Stop-Signal gesendet.[GW22] In der minimalen Konfiguration werden Master und Slave über die bidirektionalen Busleitungen Serial Data (SDA) und Serial Clock (SCL) verbunden, die über Pull-up-Widerstände an die Versorgungsspannung angeschlossen sind. Weitere Geräte können durch Verbindung ihrer SDA- und SCL-Anschlüsse mit den entsprechenden Busleitungen an den Bus angeschlossen werden.[MS21] In diesem Projekt dient der Arduino als Master und ein zusätzlich angeschlossenes Organic Light Emitting Diode (OLED)-Bildschirm als Slave.

4.3.2. USB

Das Board kann über einen Mini Universal Serial Bus (MUSB) mit einem Computer verbunden werden, um es zu programmieren oder Daten zu übertragen. Die Datenübertragungsrate beträgt dabei 12 Mbit/s. Außerdem kann der Arduino über diesen MUSB-Port Strom beziehen.

4.3.3. Bluetooth®

Die Bluetooth-Verbindung kann als drahtloser Kommunikationsweg eingesetzt werden. Das Bluetooth-Protokoll hat eine Übertragungsrate von 2 Megabit pro Sekunde (Mbps) und eine Sendeleistung von +8 Dezibel Milliwatt (dBm). Die Empfindlichkeit beträgt dabei -95 dBm. Des weiteren verbraucht diese Verbindung im Sendebetrieb 4,8 mA und 4,6 mA im Empfangsbetrieb. Das Bluetooth-Modul ist kompatibel mit mehreren Protokollen, unter anderem mit dem *Thread-Protokoll* und dem *Zigbee-Protokoll*. [Ard25] [Nor24a]

4.3.4. Weitere Kommunikationsschnittstellen

[Ard25]

- **NFC-A-Tag:** Near Field Communication (NFC) ist eine zusätzliche Funktion zur drahtlosen Kommunikation über kurze Distanzen. Zudem besitzt der NFC-A-Tag die Funktionen, sich in einen Bereitschaftsmodus versetzen zu lassen, dass durch ein NFC-fähiges Gerät dann initiiert werden kann. Außerdem unterstützt es *touch-to-pair*, diese Funktion ermöglicht eine Kopplung mit anderen NFC-fähigen Geräten durch Berührung.
- **Arm CryptoCell CC310 Security Subsystem:** Für die Durchführung kryptografischer Operationen und Sicherheitsaufgaben. [Nor24c]
- **QSPI/SPI/TWI/I²S/PDM/QDEC:** Verschiedene weitere serielle Kommunikationsschnittstellen, die für den Datenaustausch verwendet werden können.
- **EasyDMA:** Direkt Memory Access (DMA) ist für die Übertragung von Daten zwischen verschiedenen Speicherbereichen, ohne dabei die CPU zu belasten. [GW22]
- **Analog to Digital Converter (ADC):** Wandelt analoge Eingangssignale in digitale Daten um. Der Wandler hat eine Auflösung von 12 Bit und eine maximale Abtastrate von 200 Kilosamples pro Sekunde (ksps).
- **128-Bit-AES/ECB/CCM/AAR-Co-Prozessor:** Co-Prozessor für kryptografische Operationen, der auf dem Advanced Encryption Standard (AES) basiert. Dieser unterstützt verschiedene Betriebsmodi wie Electronic Codebook (ECB), Counter with CBC-MAC (CCM) und Automatic Address Recognition (AAR). [Nor24b]
- **Quad-SPI-Schnittstelle 32 MHz:** SPI-Schnittstelle, die eine maximale Taktrate von 32 MHz unterstützt. Quad-SPI ermöglicht es, Daten schneller als die herkömmliches SPI zu übertragen, indem es vier Datenleitungen verwendet. [Nor23]

4.3.5. Digitale Ein- und Ausgangspins

Das Board verfügt über 14 digitale Ein- und Ausgangspins. Die digitalen Pins können nur zwei Zustände, nach dem Binär-System lesen: wenn ein Spannungssignal vorliegt

und wenn kein Signal vorhanden ist (0 oder 1). Einige der Pins sind zudem zur Pulswweitenmodulation fähig (D3, D5, D6, D9, D10). Außerdem sind die digitalen Pins D11 und D12 als Master-Output-Slave-Input (MOSI) und als Master-Input-Slave-Output (MISO), in einer Serial-Peripheral-Interface (SPI) Kommunikation einsetzbar.[Ard25]

4.3.6. Analoge Eingangspins

Die Platine hat zusätzlich 8 analoge Eingangspins (A0-A7), die wiederum als Analog-Digital-Wandler (ADC) verwendet werden können. Außerdem sind diese Pins als digitale Ein-/Ausgangspins konfigurierbar. Der Pin A0 kann zudem als Digital-Analog-Wandler (DAC) konfiguriert werden. Die beiden Pins A4 und A5 können außerdem für die I2C-Kommunikation verwendet werden. Dabei fungiert A4 als Datenleitung (SDA), während A5 als Taktleitung (SCL) fungiert.[Ard25]

4.3.7. Weitere Pins

- **+3,3 V**: Erzeugt interne Stromquelle im Gerät und wird als Referenzspannung verwendet.
- **VIN**: Stromversorgung
- **5V**: Gibt 5V an die externen Komponenten ab.
- **RST-Pin**: Dient zum Zurücksetzen des Arduinos.
- **AREF-Pin**: Liefert die Spannungsreferenz, die der Mikrocontroller zur Zeit verwendet.
[Ard25]

4.3.8. Reset-Knopf

Der Arduino Nano 33 BLE Sense verfügt über einen Reset-Knopf, welcher das Neustarten der Platine, das Aktivieren des Bootloader-Modus und das Beheben von Problemen (z.B. bei fehlerhaften Programmen) ermöglicht.

Der Reset-Knopf befindet sich auf der Oberseite der Platine, in unmittelbarer Nähe zum USB-Anschluss. Je nach Anzahl und Geschwindigkeit der Tastendrucke führt er unterschiedliche Aktionen aus. Bei einem einzelnen Druck auf die Taste wird ein Systemneustart veranlasst, dabei wird die Stromversorgung kurzzeitig unterbrochen, sodass die Platine und das darauf laufende Programm neu startet. Wenn der Reset-Knopf zweimal schnell hintereinander, innerhalb einer Sekunde gedrückt wird, wird der Bootloader-Modus aktiviert. Dieser ist wichtig, um neue Programme hochzuladen oder das Board wiederherzustellen, falls es durch fehlerhafte Programme blockiert wurde. Die Aktivierung des Bootloader-Modus ist daran zu erkennen, dass die eingebaute LED schnell blinkt.[Ard25]

4.3.9. LED-Lampen

Im Arduino selbst sind 3 LED's verbaut, die auch alle programmiert werden können. Diese sind vor allem für die Überprüfung der Sensorik oder Softwareprogrammen nützlich, denn sie können je nach Benutzeraktion oder Programmcode blinken, erlöschen oder dauerhaft leuchten. Zu den LED-Lampen gehören:

- Programmierbare Power-LED (grün): Zeigt an, dass das Arduino-Board eingeschaltet ist.
- Programmierbare LED (orange)
- Programmierbare RGB-LED
[Ard25]

4.4. Hinweis Arduino 33 BLE Sense Lite

Der Arduino Nano 33 BLE Sense Lite ist eine komprimierte Variante vom ursprünglichen Arduino Nano 33 BLE Sense, welcher zusätzlich noch über einen Temperatur- und Feuchtigkeitssensor verfügt. Der Lite hat stattdessen einen Drucksensor integriert, über welchem auch die Temperatur gemessen werden kann, jedoch nicht die Feuchtigkeit.[PA22]

4.5. Bezugsquellen

Als Bezugsquellen dienen vor allem Datenblätter der Hersteller. Die meisten Informationen konnten dem Datenblatt des Arduino entnommen werden, jedoch ist dazu anzumerken, dass sich dieses Datenblatt auf den Arduino 33 BLE Sense bezieht und nicht auf den Arduino 33 BLE Sense *Lite*. Speziell zum *Lite* gibt es jedoch kein Datenblatt, so wurde das Datenblatt vom Arduino 33 BLE Sense herangezogen. Dies stellt sonst kein Problem dar, da sich, wie in 4.4 beschrieben, nur um eine komprimierte Version des Arduino 33 BLE Sense handelt.

Part III.

Arduino Nano 33 BLE Sense - External Sensors and Actors

5. Schrittmotor NEMA 17

In diesem Kapitel folgt eine grundsätzliche Beschreibung eines Schrittmotors. Darauf folgt die Erläuterungen zum Aufbau und Funktionsweise eines Schrittmotors sowie die Aufzählung verschiedener Grundbauarten. Nachfolgend werden Ursachen und Lösungen zum Verhindern von Schrittfehlern und der verwendete Schrittmotor genauer erläutert.

5.1. Beschreibung eines Schrittmotors

Ein Schrittmotor ist ein Elektromotor, der sich für präzise Positionierungsaufgaben eignet. Im Gegenteil zu anderen Elektromotoren wird bei einem Schrittmotor keine Positionsmessung oder Positionsregelung benötigt. Andere mechanische Antriebssysteme benötigen einen geschlossenen Regelkreis und mechanische Bremsen um Drehzahl und Position einzuhalten. Der Motor führt durch die Rotation des Rotors diskrete Schritte aus, wobei jeder Steuerimpuls eine Verschiebung um einen konstanten Winkel bewirkt. Mit diesem Motor ist eine erreichbare Positioniergenauigkeit von $0,1^\circ$ möglich. Diese Art von Elektromotor wird beispielsweise in Druckern oder Scanner, aber auch im Kraftfahrzeugbereich verwendet. Im Kraftfahrzeugbereich werden die Schrittmotoren zur Spiegelverstellung sowie der Sitzverstellung verwendet. Das maximal erreichbare Drehmoment eines Schrittmotors liegt bei zwei Newtonmeter und die maximal erreichbare Drehzahl bei ca. 2000 Umdrehung pro Minute. Der Vorteil eines Schrittmotors ist die Wartungsfreiheit, da der Rotor keine Wicklungen hat. [Bab23][Hag21][Ber18][SK21]

5.1.1. Aufbau und Funktionsweise eines Schrittmotors

Der Schrittmotor besteht aus einem Stator, einem Rotor ohne Wicklungen und einer Steuerelektronik. Diese Steuerelektronik setzt sich zusammen aus einer Treiberstufe und der eigentlichen Steuerung. Bei dem Stator handelt es sich um den feststehenden äußeren Teil und bei dem Rotor um den beweglichen inneren Teil, wie in Abbildung 5.1 zu erkennen ist. Im Stator sind Spulen verbaut, die von einem Strom durchflossen werden. Hierdurch entsteht ein magnetisches Feld. Da der Rotor magnetisch ist, folgt er dem Magnetfeld des Stators. Soll eine Bewegung hervorgerufen werden, werden einzelne Wicklungsstränge ein- aus- oder umgeschaltet. Durch diesen Vorgang wird ein rotierendes Magnetfeld erzeugt. Das erzeugte Magnetfeld zieht den Rotor an. Der Rotor wird bei jedem Puls (Takt) um einen Winkelschritt weitergeschaltet. Die Anzahl der Polpaare im Stator geben die Anzahl der Schritte vor. Die Drehzahl und die Drehrichtung hängt von der Reihenfolge und der Häufigkeit der Stromimpulse ab. Es gibt drei verschiedene Betriebsarten, die abhängig von der Genauigkeit und der Drehzahl sind. Im Vollschrittbetrieb werden alle Polpaare bestromt. In dem Halbschrittbetrieb wird die Schrittzahl des Motors verdoppelt, wodurch sich die Positionsauflösung im Gegensatz zum Vollschrittbetrieb verdoppelt. Allerdings wird in diesem Betrieb das Drehmoment reduziert. In dem Mikroschrittbetrieb bewegt sich der Rotor in sehr kleinen Schritten. Hierdurch wird eine hohe Positioniergenauigkeit und ein ruhiger Lauf erreicht, da die Ströme und das Drehmoment in kleineren Schritten verändert werden. Bei einer zu hohen Schrittfrequenz kann ein Schrittverlust entstehen. Bei einem Schrittverlust überspringt der Motor einzelne Winkelschritte und landet in einer vorherigen oder nächsten Position gleicher Phase. Durch die offene Steuerkette werden die Verluste nicht erkannt.[Hag21][Ber18][SK21]

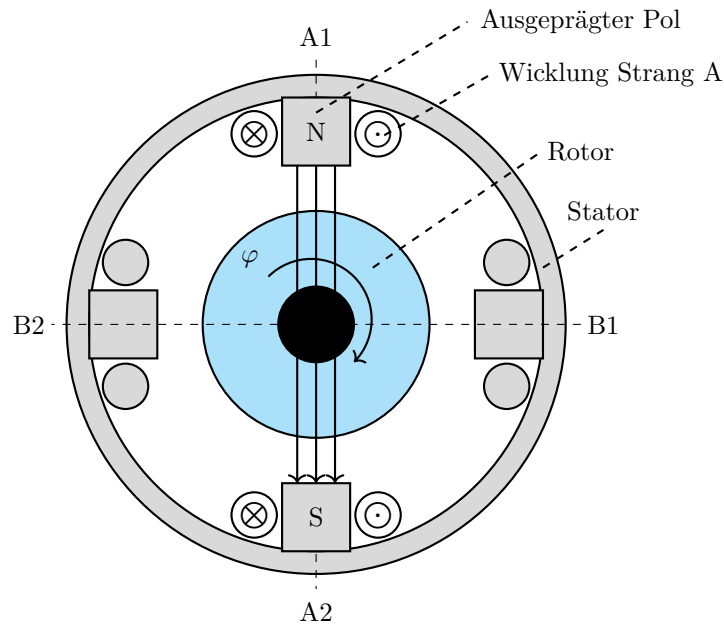


Figure 5.1.: Prinzipieller Aufbau Schrittmotor (2-phasig) [Hag21]

5.1.2. Schrittmotor Bauformen

Wie bei anderen Elektromotoren gibt es auch bei dem Schrittmotor verschiedene Grundbauarten.

- Permanentmagnetereger-Schrittmotor (PM-Schrittmotor)
- Reluktanzschrittmotor (VR-Schrittmotor)
- Hybridschrittmotor

Der **permanentmagnetische Schrittmotor** hat einen Permanentmagneten in dem Rotor verbaut. Hierbei stellt sich der permanentmagnetische Rotor immer so, dass der Nordpol des Rotors dem Nordpol des Statorfeldes gegenüber liegt und der Südpol des Rotors dem Südpol des Statorfeldes. In dieser Ausrichtung ziehen sich die Pole gegenseitig an. Die Drehrichtung des Rotors hängt von der Fließrichtung des Stromes ab. Der permanentmagnetische Schrittmotor entwickelt im ausgeschalteten Zustand ein Drehmoment zur Selbsthaltung. Dies ist aufgrund des permanentmagnetischen Rotors möglich. Bei diesem Drehmoment handelt es sich um das höchste Drehmoment, das auf die Welle des Motors übertragen werden kann, ohne dass diese sich in eine rotierende Bewegung versetzt. Zu dieser Art von Schrittmotoren gehören beispielsweise der Klauenpol-Schrittmotor und der Scheibenmagnet-Schrittmotor. Bei der zweiten Bauart handelt es sich um den **Reluktanzschrittmotor**. Bei dieser Bauart besteht der Rotor aus einem weichmagnetischen Material und besitzt eine gezahnte Form. So lange der Schrittmotor von keinem Strom durchflossen wird, entsteht kein Magnetfeld. Sobald der Motor in Betrieb genommen wird, entsteht ein magnetischer Fluss innerhalb des Rotors. Wird nun eine Wicklung erregt, wird der nächste Zahn des Rotors angezogen. Dadurch, dass die Rotorzähne ungleich der Polteilung sind, kann das System unendlich lange fortgesetzt werden. Die Anzahl der Schritte und

die Genauigkeit des Reluktanzschrittmotors ist abhängig von der Anzahl der Zähne auf dem Rotor. Aus technischer Sicht sind mit dieser Bauart Schrittwinkel unter 1° möglich. Damit die Drehrichtung verändert werden kann, sind mindestens zwei Strangwicklungen nötig. Bei den **Hybridschrittmotoren**, auch bekannt als Hybridschrittmotoren handelt es sich um eine Kombination aus dem Reluktanzschrittmotor und dem Permanentmagneterreger-Schrittmotor. Durch diese Kombination aus den beiden Schrittmotoren werden die Vorteile aus der kleinen Schrittweite, dem hohen Drehmoment und dem Selbsthaltemoment genutzt. Bei dem Hybridschrittmotor besteht der Rotor aus zwei um eine halbe Zahnteilung versetzten weichmagnetischen Polrädern, die eine zahnförmige Form haben. Bei den beiden Polrädern bildet das eine Polrad den Nordpol und das zweite Polrad den Südpol. Zwischen den beiden Polrädern befindet sich ein Permanentmagnet. Anders als bei anderen Schrittmotoren wird der Rotor bei dieser Bauart axial magnetisiert. Damit ein kleiner Schrittwinkel möglich ist, haben die Statorpole ebenfalls eine zahnförmige Form. Die Ausrichtung des Rotors ist abhängig von der Stromrichtung und wird durch den minimalen Widerstand bestimmt, der sich aus dem Stromfluss durch die einzelnen Stränge ergibt. Wird ein besonders kleiner Schrittwinkel benötigt, kann dies durch Erhöhung der Zähnezahl erreicht werden. [SK21] [Hag21] [Bab23]

5.1.3. Betriebsarten unipolar und bipolar

Neben den oben bereits genannten Betriebsarten, kann außerdem zwischen dem Unipolarbetrieb und dem Bipolarbetrieb unterschieden werden. Der große Unterschied zwischen den beiden Betriebsarten besteht darin, dass in dem Unipolarbetrieb der Strom in eine Richtung fließt. Bei dem Bipolarbetrieb hingegen fließt der Strom in beide Richtungen. Dies ist möglich, da jeder Wicklungsstrang über eine Vollbrücke gespeist wird. Ein weiterer Unterschied besteht in der Schaltung der Zweige, durch die ein Gleichstrom fließt. In dem Unipolarbetrieb werden die beiden Zweige in Reihe geschaltet. Jeder Wicklungsstrang wird mit zwei Drähten parallel gewickelt. Sind die beiden Wicklungsstränge in dem Bipolarbetrieb parallel gewickelt, müssen die Zweige parallelgeschaltet werden. Im Bipolarbetrieb kann ein höherer Wirkungsgrad erzielt werden, wo hingegen der Unipolarbetrieb eine deutlich einfachere Schaltung aufweist. [SK21]

5.1.4. Mikroschrittverfahren

Mit dem Mikroschrittverfahren können Zwischenschritte und Mikroschritte erzeugt werden. Es bietet die Fähigkeit, Geräusche und Vibrationen zu minimieren sowie die Auflösung der Umdrehung zu erhöhen. Um Zwischenschritte zu erreichen, müssen beide Wicklungen eines Schrittmotors gleichzeitig mit Strom versorgt werden. Mit dem Sinus/Cosinus-Mikroschrittverfahren können die Mikroschrittverfahren realisiert werden, indem der Strom in jeder Phase des Motors sinusförmig variiert. Durch die Anwendung sinusförmiger Ströme auf die Motorwicklungen wird eine feinere und gleichmäßigere Bewegung erzielt, was zur einer präziseren Steuerung und reduzierten mechanischen Vibrationen führt. [Mar24]

Ein Mikroschritt-Treiber unterteilt die Motorschrittwinkel in vier oder mehr Teil. Es handelt sich um einen Controller, der die Methode der Stromreglung verwendet, um die Mikroschritte zu erzeugen. Der Controller bietet außerdem den Vorteil einer erhöhten Flexibilität bei der Integration der Strombegrenzung und der Anpassung der Antriebscharakteristik als Reaktion auf Änderungen der Systemdynamik. Die Anzahl der Unterteilungen ist einstellbar, was eine präzise Steuerung und Anpassung an spezifische Anforderungen ermöglicht. [Bab23]

5.2. Schrittverluste verhindern

Um mögliche Schrittverlusten einzugrenzen und oder sie zu verhindern, wird in diesem Kapitel beschrieben, wie die Ursachen für Schrittverluste oder Stillstand methodisch zu ermitteln sind. [Fau20]

5.2.1. Auswahl des Schrittmotors

Zunächst muss ein passender Motor für die Anwendung gewählt werden. Dabei sollten folgende grundlegende Regeln erfüllt sein:

- Motorauswahl durch Höchstwerte für Drehmoment und Drehzahl (Worst-Case-Szenario)
- Verwendung eines Sicherheitsaufschlag von 30 % auf die Drehmoment-Drehzahl-Kennlinie(Kippmoment)
- Sicherstellen, dass externe Ereignisse die Anwendung nicht blockieren können

Sind die geforderten Drehmomente bei den jeweiligen Drehzahlen, den Motorspezifikation entsprechend, dann sind keine Probleme zu erwarten. Ist der Motor zu schwach gewählt und die Anwendung fordert mehr Leistung als der Motor abgeben kann, so bleibt der Motor stehen. Der nächste Schritt ist die Durchführung von Testdurchläufen. Es soll im Betrieb überprüft werden, ob Schrittverluste auftreten. Schrittmotoren verlieren konstruktionsbedingt nicht nur einen einzigen Schritt. Bei geringen Drehzahlen verliert der Motor ein Vielfaches von vier Schritten.[Fau20]

5.2.2. Betriebsart

In diesem Abschnitt werden je nach Betriebsart mögliche Ursachen erläutert, falls der Schrittmotor bei den Tests versagt.

Start-Stopp-Betrieb

Der Motor ist mit der Last fest verbunden und wird mit konstanter Drehzahl betrieben. Innerhalb des ersten Schrittes muss der Motor auf die vorgegebene Frequenz beschleunigen wie in Abbildung 5.2 zu sehen ist.[Fau20]

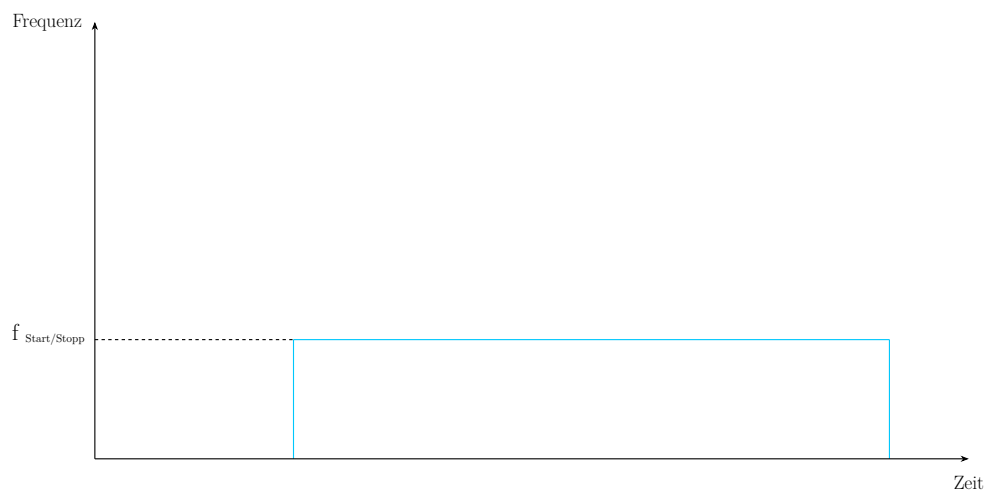


Figure 5.2.: Start-Stopp Frequenz [Fau20]

Fehlerbild: Motor läuft nicht an (siehe Ursachen und Lösungen aus Tabelle 5.1)

Ursachen	Lösungen
Last zu Hoch	Falscher Motor, größeren Motor wählen
Frequenz zu hoch	Frequenz reduzieren
Pendelt der Motor von Links nach Rechts	Es könnte eine Phase unterbrochen oder nicht angeschlossen sein, dies muss repariert werden
Phasenstrom passt nicht	Phasenstrom erhöhen

Table 5.1.: Ursachen und Lösungen: Motor läuft nicht an [Fau20]

Beschleunigung und Rampenprofil (Trapezförmig)

Der Motor kann mit einer im Controller vorgegebenen Beschleunigungsrate bis auf die Maximalfrequenz beschleunigen wie in Abbildung 5.3 zu sehen ist.[Fau20]

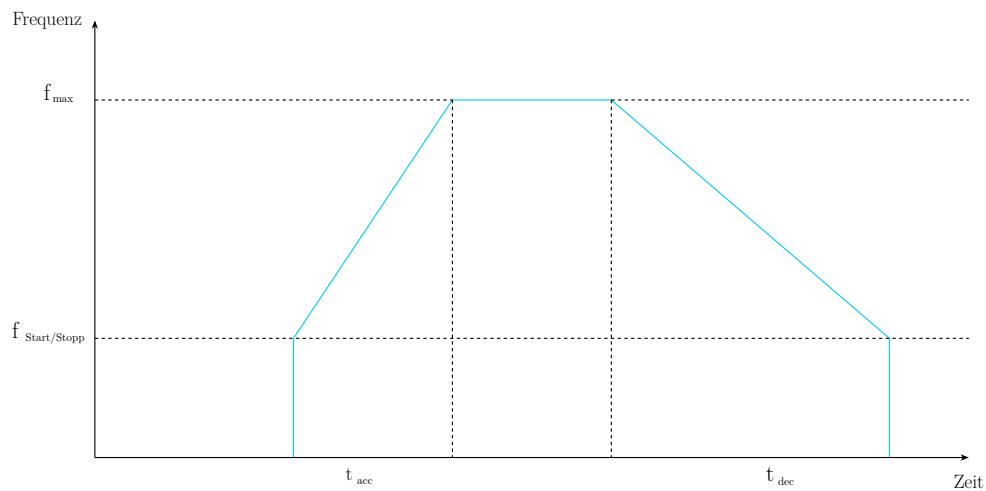


Figure 5.3.: Trapezförmiges Geschwindigkeitsprofil [Fau20]

Fehlerbild: Motor beendet die Beschleunigungsrampe nicht (siehe Ursachen und Lösungen aus Tabelle 5.2)

Ursachen	Lösungen
Motor bleibt bei der Resonanzfrequenz hängen	• Beschleunigung erhöhen, um die Resonanzfrequenz schneller zu durchlaufen • Start-Stopp Frequenz über dem Resonanzpunkt wählen • Halbschritt- oder Mikroschrittbetrieb verwenden • Mechanische Dämpfung vorsehen
Falsche Einstellung von Versorgungsspannung oder Strom zu gering	• Spannung oder Strom erhöhen • Motor mit geringerer Impedanz testen • Stromregelung verwenden
Maximaldrehzahl zu hoch	• Maximaldrehzahl reduzieren • Beschleunigungsrampe abflachen
Schlechte Vorgabe der Beschleunigungsrampe durch Elektronik	• Anderen Controller verwenden

Table 5.2.: Ursachen und Lösungen: Motor beendet die Beschleunigungsrampe nicht [Fau20]

Fehlerbild: Motor beschleunigt bis zur Enddrehzahl und bleibt stehen, sobald eine konstante Drehzahl erreicht ist (siehe Ursachen und Lösungen aus Tabelle 5.3)

Ursachen	Lösungen
Motor wird an Leistungsgrenze betrieben und bleibt stehen aufgrund zu hoher Beschleunigung	• Ruckeln verringern durch geringere Beschleunigungsrate oder durch unterschiedliche Beschleunigungsrampen, erst steil dann flacher • Drehmoment erhöhen • Motor im Mikroschrittbetrieb betreiben • Mechanische Dämpfung vorsehen

Table 5.3.: Ursachen und Lösungen: Motor beschleunigt bis zur Enddrehzahl und bleibt stehen, sobald eine konstante Drehzahl erreicht ist. [Fau20]

5.2.3. Externe Ereignisse

Lastrückkopplung

„Manchmal wird der vom Motor angetriebene Mechanismus/Last während der Bewegung „aufgezogen“ und gibt diese Energie wieder an den Motor zurück, wenn die Ströme ausgeschaltet werden. Der Mechanismus könnte z.B. ein Untersetzungsgetriebe sein.“ [Fau20] Wird diese Energie an den Motor zurück geleitet, kann es passieren, dass der Motor sich um einen Winkel verdreht, der mehr als einem Schritt entspricht. Dabei kann es passieren, dass der Motor nicht ausreichend Drehmoment entwickelt und nicht oder erst nach 4 Vollschritten anläuft. [Fau20]

Lösungen:

- Die Kommutierung so programmieren, dass Wert und Polarität vor dem Abschalten gespeichert und beim Wiedereinschalten verwenden werden kann.
- Nicht vollständig den Strom abschalten, sondern bei Motorstillstand einen reduzierten *Stand-By* Strom aufrechterhalten

Erhöhung der Nutzlast mit der Zeit

„Manchmal läuft der Motor für eine lange Zeit störungsfrei und viel später treten die ersten Schritverluste auf. In diesem Fall ist es sehr wahrscheinlich, dass die Last, die der Motor „sieht“, sich geändert hat. Das kann auf Verschleiß der Motorlager oder ein externes Ereignis zurückzuführen sein.“[Fau20]

Lösungen:

- Prüfen ob ein externes Ereignis durch Veränderung des Mechanismus vorliegt.
- Prüfen ob Lagerverschleiß vorhanden ist. Verwendung von Kugellager erhöhen die Lebensdauer des Motors.
- Verwendung von Schmiermittel um Reibung zu verhindern.

5.3. Beschreibung des verwendeten Schrittmotors

Der hier verwendete NEMA 17 Schrittmotor ist ein bipolarer Schrittmotor. "NEMA" steht hier für die Norm des US-amerikanischen "National Electrical Manufacturers Association". Die 17 in der Modellbezeichnung steht für die Flanschgröße, welche 1,7 x 1,7 Zoll beträgt. Der NEMA 17 besitzt keinen integrierten Treiber, sondern muss über einen externen Treiber wie den DRV8825 angesteuert werden. In Verbindung mit dem Treiber unterstützt der Schrittmotor bis zu 1/32 Mikroschritte.

Der NEMA 17 Schrittmotor findet vor allem in 3D-Druckern, CNC-Fräsen, Kameraschlitten oder Robotikachsen Anwendung. [JOY19]

5.4. Spezifikation

In Tabelle Table 5.4 sind die Spezifikationen des NEMA 17 aufgelistet.

Spezifikation	Wert
Wellenmaße	4,5 x 22 mm (Einzelwelle)
Anschluss	4-Pol Buchse
Schritte pro Umdrehung	200
Abmessungen	42 x 42 x 42 mm
Haltemoment	0,45 Nm
Nennspannung	3,3 V
Nennstrom	1,5 A
Schrittwinkel	1,8°
Anzahl der Phasen	2
Phasenwiderstand	2,2 Ohm
Phaseninduktivität	5 mH
s Isolationswiderstand	500 V DC
Isolationsklasse	B (130°)
Trägheitsmoment	57 g · cm ²
Rastmoment	0,015 Nm
zuläss. Temperaturbereich	-10 °C bis 50 °C

Table 5.4.: Spezifikationen des NEMA 17 Schrittmotors [JOY19]

6. Schrittmotortreiber DRV8825

In diesem Kapitel wird beschrieben, was ein Motortreiber ist, welche Aufgaben er erfüllt und wie man den DRV8825 mit dem Arduino Nano 33 BLE Sense verwendet.

6.1. Allgemeine Beschreibung eines Motortreibers

Ein Motortreiber ist ein Elektronikbaustein, der Mikrocontrollern wie dem Arduino Nano 33 BLE Sense ermöglicht elektrische Motoren anzusteuern. Dabei kann es sich zum Beispiel um Gleichstrommotoren oder Schrittmotoren handeln.

Motortreiber haben mehrere Funktionen. Zum einen verstärken sie die Spannung und den Strom, da Mikrocontroller meist zu wenig Leistung liefern, um den Motor direkt zu betreiben. Zum anderen sorgen Motortreiber dafür, dass man die Drehrichtung und die Drehzahl regeln kann. Außerdem haben viele Motortreiber auch eine Schutzfunktion, die vor Überstrom, Überhitzung oder Kurzschluss schützen. [Tex14]

6.2. Spezifische Beschreibung des Schrittmotortreibers DRV8825

Der DRV8825 ist ein Motortreiber, der vor für bipolare Schrittmotoren wie hier der NEMA 17-04 eingesetzt wird. Er wird in Druckern, CNC-Maschinen oder in der Robotik verwendet.

Der Motor lässt sich über den DRV8825 von Vollschritt bis 1/32-Schritt einstellen. Er verfügt über einen konfigurierbaren Abklingmodus (tritt bei Erreichen des Stromlimits ein), sodass langsames, schnelles oder gemischtes Abklingen verwendet werden kann. Außerdem besitzt er einen stromsparenden Schlafmodus, der die interne Schaltung abschaltet und den Ruhestromverbrauch minimiert.

Der DRV8825 verfügt über Schutzfunktionen für Überstrom, Kurzschluss, Unterspannung und Übertemperatur. Außerdem werden Fehler über den nFAULT-Pin signalisiert. [Tex14]

6.2.1. Anschluss des Treibers mit dem Arduino Nano 33 BLE Sense

Tabelle Table 6.1 kann entnommen werden, wie der DRV8825 an den Arduino Nano 33 BLE Sense angeschlossen wird. Tabelle Table 6.2 zeigt, wie der DRV8825 an den Schrittmotor NEMA 17-04 angeschlossen wird.

DRV8825	Arduino Nano 33 BLE Sense
STEP	z.B. D3
DIR	z.B. D4
EN	GND
nRESET	HIGH (z.B. an 5V)
nSLEEP	HIGH (z.B. an 5V)
GND	GND
VDD	5V

Table 6.1.: Pin-Belegung für den Anschluss des Treibers an den Arduino [Tex11]

DRV8825	Schrittmotor
AOUT1	Spule A, Ende 1
AOUT2	Spule A, Ende 2
BOUT1	Spule B, Ende 1
BOUT2	Spule B, Ende 2

Table 6.2.: Pin-Belegung für den Anschluss des Treibers an den Schrittmotor

6.3. Spezifikationen

Die Spezifikationen sind in Tabelle Table 6.3 und Table 6.4 zu sehen.

Spezifikation	Wert
Versorgungsspannung	-0,3 V bis 47 V
Digitale Eingangsspannung	-0,5 V bis 7 V
Referenzspannung	-0,3 V bis 4 V
Motorstrom	bis zu 2,5 A
Temperaturbereich	-40 °C bis 150 °C

Table 6.3.: Absolute Maximalwerte [Tex11]

Spezifikation	Wert
Versorgungsspannung	8,2 V - 45 V
Referenzspannung	1 V bis 3,5 V
Motorstrom	0 A bis 2,5 A

Table 6.4.: Empfohlene Betriebsbedingungen [Tex11]

6.4. Kalibrierung

Da der Treiber den Strom durch den Schrittmotor regelt ist die Kalibrierung des DRV8825 erforderlich, um zu vermeiden, dass der Schrittmotor überhitzt oder der Treiber überlastet.

Für die Kalibrierung wird ein Multimeter, ein Schraubendreher für das Potentiometer, der GND-Pin des Arduino und der VREF-Messpunkt (Metallplättchen am Potentiometer) benötigt.

Im ersten Schritt ist es wichtig, dass der Schrittmotor nicht angeschlossen ist. Als nächstes wird die Stromversorgung eingeschaltet. Nun wird das Multimeter verwendet, dabei wird der Pluspol am VREF-Messpunkt verbunden und der Minuspol am GND-Anschluss des Arduino. Aus dem abgelesenen Wert kann nun die Zielspannung für VREF berechnet werden und dann am Potentiometer mit dem Schraubendreher eingestellt werden. Dabei ist darauf zu achten das Potentiometer nicht zu überdrehen, da es empfindlich ist.

Nach dem Kalibrieren des Treibers kann der Schrittmotor angeschlossen werden.

6.5. Einfaches Beispiel mit Code

```
// pin assignment
const int stepPin = 3;           //STEP
const int dirPin = 4;            //DIR
const int enablePin = 5;         //optional: nENBL (LOW = aktiv)
```

```
void setup() {
  pinMode(stepPin, OUTPUT);
  pinMode(dirPin, OUTPUT);
  pinMode(enablePin, OUTPUT);

  digitalWrite(enablePin, LOW);    // enable driver
  digitalWrite(dirPin, HIGH);
  // set rotation direction (HIGH or LOW)
}

void loop() {
  // generate one step:
  digitalWrite(stepPin, HIGH);
  delayMicroseconds(1000);
  // step pulse duration (lower = faster)
  digitalWrite(stepPin, LOW);
  delayMicroseconds(1000);
  // pause time (depending on desired speed)
}
```


7. Geschwindigkeitssensor LM393

7.1. Allgemeine Beschreibung des Sensors

Der LM393 ist ein einfacher und kostengünstiger Geschwindigkeitssensor, der nach dem Prinzip der optischen Lichtschranke funktioniert. Er besteht aus einem LM393-Komparator, einem Infrarot-Sender-Empfänger-Paar und einem digitalen Ausgang. Der Sensor kann in Kombination mit einer auf einer rotierenden Welle angebrachten Lochscheibe verwendet werden. [Tex25] [Par20]

7.2. Spezifische Beschreibung des Sensors

Der LM393 Geschwindigkeitssensor besteht aus folgenden Komponenten:

- Infrarot-LED (Sender)
- Fototransistor (Empfänger)
- Lichtschranken-Nut
- LM393 Komparator
- Status-LED
- PCB mit Pins
- Lochscheibe

Wie diese Komponenten die Funktion des Sensors ermöglichen, wird im Folgenden erklärt.

7.2.1. Funktionsweise der Lichtschranke

Die Infrarot-LED und der Fototransistor bilden eine optoelektronische Lichtschranke und sind sich gegenüber auf dem Sensor-PCB angebracht. Die beiden Komponenten sind durch die Lichtschranken-Nut getrennt, welche 5 mm breit ist. In dieser Nut rotiert die Lochscheibe, welche auf der Motorwelle montiert ist und mit 20 Löchern ausgestattet ist. Die Infrarot-LED dient als Sender der Lichtschranke und emittiert kontinuierlich Licht im infraroten Bereich (940 nm) in Richtung des Empfängers. Der Fototransistor ist der Empfänger. Dieser Fototransistor besteht aus einem Halbleitermaterial. Trifft das Infrarotlicht des Senders, durch ein Loch der Lochscheibe auf den Empfänger, so werden dort Elektronen-Loch-Paare erzeugt. Dadurch steigt der Stromfluss durch den Fototransistor. Wird das Infrarotlicht durch die Lochscheibe blockiert sinkt der Strom im Fototransistor.

Die hier verwendete Lochscheibe weist 20 Löcher auf, somit werden pro Umdrehung 20 Signale erzeugt. Da der Sensor aber auf Signaländerungen reagiert werden pro Loch zwei Impulse erzeugt, es werden also 40 Impulse pro Umdrehung gezählt. [Par20]

7.2.2. Signalverarbeitung mit dem LM393 Komparator

Der LM393 ist ein Komparator, der zwei Analoge Spannungen vergleicht. Eingang A wird mit einer Referenzspannung beschaltet. Eingang B ist mit dem Fototransistor verbunden. Wenn Infrarotlicht auf dem Empfänger auftrifft (durch Loch der Lochscheibe) ändert sich die Spannung am Eingang B. Wenn die Spannung im Fototransistor durch den Lichteinfall höher wird als die Referenzspannung, dann schaltet der Komparator den digitalen Ausgang D0 auf LOW. Wenn der Lichteinfall blockiert ist und somit die Spannung im Fototransistor niedriger ist als die Referenzspannung, dann schaltet der Komparator den digitalen Ausgang D0 auf HIGH. Der LM393 Komparator wird also dazu genutzt kleine Änderungen im Analogsignal zu verstärken und in ein klares digitales Rechtecksignal umzuwandeln.

Am digitalen Ausgang D0 liegt das durch den LM393 erzeugte Signal an. Dieses kann direkt mit einem Mikrocontroller, in diesem Fall einem Arduino Nano 33 BLE Sense verbunden und ausgewertet werden. [Par20]

7.2.3. Anschluss des Sensors mit dem Arduino Nano 33 BLE Sense

In Tabelle Table 7.1 ist gezeigt wie der Sensor korrekt an den Arduino Nano 33 BLE Sense angeschlossen wird:

Arduino	LM393
5V	VCC
GND	GND
Pin 2	D0

Table 7.1.: Anschluss Speedsensor LM393 an Arduino Nano 33 BLE Sense [SIM24]

7.3. Spezifikationen

Spezifikation	Wert
Modell	Speedsensor LM393 mit Lochscheibe
Versorgungsspannung	3,3 V - 5 V DC
Signalausgabe	Digital
Nutbreite (Lichtschranke)	5 mm
PCB-Abmessungen	32 mm x 14 mm
Lochscheiben-Durchmesser	25,5 mm (20 Löcher)
Besonderheiten	LED-Anzeige, Montageloch

Table 7.2.: Spezifikationen des Speedsensor LM393 [SIM20]

7.4. Bibliothek

Für die Verwendung des Sensors mit dem Arduino Nano 33 BLE Sense wird die TimerOne-Bibliothek benötigt. Die TimerOne-Bibliothek wird verwendet, um die Zeitmessung zu ermöglichen, um aus den erzeugten Impulsen des Sensors die Drehzahl zu berechnen.

7.4.1. Installation

Im Folgenden wird erklärt, wie die TimerOne Bibliothek installiert wird.

1. Öffnen der Arduino IDE
2. Klick auf Sketch, Bibliothek einbinden, Bibliotheken verwalten (Abbildung Figure 7.1)
3. In der Suchleiste TimerOne eingeben
4. Klick auf Installieren (Version aus Abbildung Figure 7.2 entnehmen)

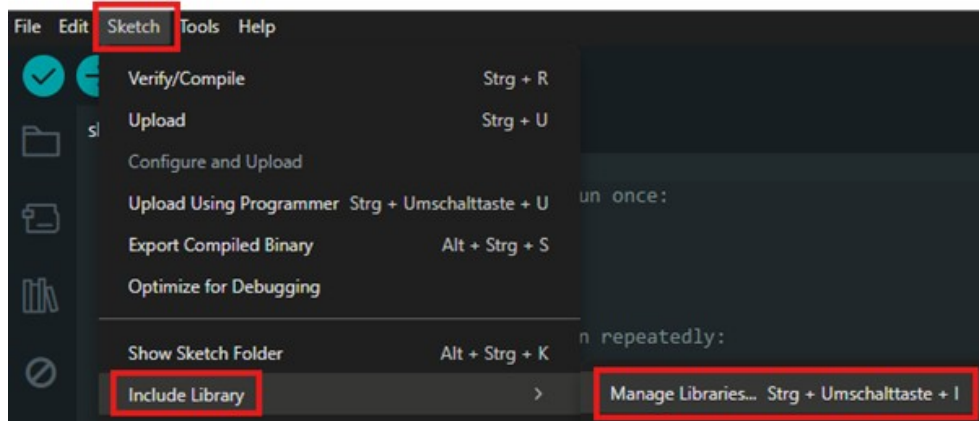


Figure 7.1.: Aufrufen der Bibliotheken in der Arduino IDE



Figure 7.2.: Installation der TimerOne Bibliothek

7.4.2. Code-Beispiel

Im Folgenden ist ein Code-Beispiel gegeben, in dem die verwendeten Funktionen erklärt werden. Für weitere Funktionen der Bibliothek kann man über Datei, Beispiele, TimerOne auf verschiedene Beispiel-Codes zugreifen (Abbildung Figure 7.3).

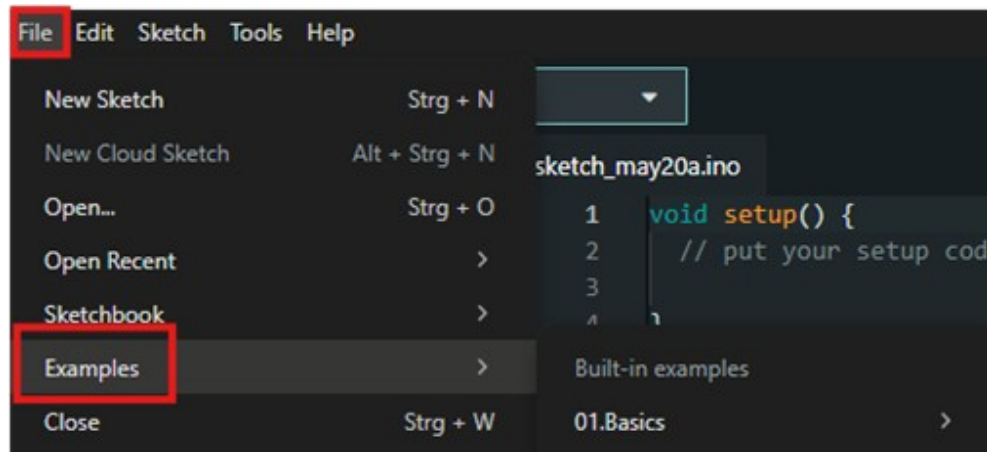


Figure 7.3.: Aufrufen der Beispiele für die TimerOne Bibliothek

7.5. Kalibrierung

Für den LM393 Geschwindigkeitssensor mit Lochscheibe ist keine spezielle Kalibrierung notwendig. Optional kann aber eine empirische Kalibrierung durchgeführt werden, indem man die Umdrehungen mit einem bekannten Drehzahlmesser vergleicht. Bei der Positionierung des Sensors muss darauf geachtet werden, dass die Lichtschranke präzise auf die Mitte der Lochscheibe ausgerichtet ist. Die Lochscheibe muss plan und fest an der Motorachse montiert werden. Außerdem muss die Variable „wheel“ im Code an die Anzahl der Löcher in der Lochscheibe angepasst werden. Da der Sensor auf Flankenwechsel reagiert, muss die Variable auf die doppelte Anzahl der Löcher gesetzt werden. Die Lochscheibe, welche für den Demonstrator verwendet wird, hat 20 Löcher, somit muss „wheel“ auf 40 gesetzt werden, wie in Abbildung xx unter Abschnitt xx zu sehen.

7.6. Einfaches Beispiel mit Code

Das Beispiel geht davon aus, dass die Lochscheibe auf einer sich rotierenden Welle befestigt ist. Mit dem folgenden Code wird die Drehzahl der Welle alle 5 Sekunden im seriellen Monitor ausgegeben.

```
// import library
#include "TimerOne.h"
#define pin 2
// required variables
int interval, wheel, counter;
unsigned long previousMicros, usInterval, calc;

void setup() {
  counter = 0; // set counter to 0
  interval = 5; // 5-second interval
  wheel = 20; // holes in encoder disk

  // scale interval to one minute
  calc = 60 / interval;
  // convert interval time to microseconds
  usInterval = interval * 1000000;
  // number of holes in encoder disk *2 equals number of signal changes
```



```

wheel = wheel * 2;

// set analog pin as input
pinMode(pin, INPUT);
// initialize timer with interval
Timer1.initialize(usInterval);
attachInterrupt(digitalPinToInterrupt(pin), count, CHANGE);
// executes 'count' when signal at pin 2 changes

// executes 'output' after each interval
Timer1.attachInterrupt(output);
Serial.begin(9600);
}

// count the encoder disk
void count() {
  if (micros() - previousMicros >= 700){
    counter++;
    previousMicros = micros();}
}

// output to serial monitor

void output() {
  Timer1.detachInterrupt(); // stop timer
  Serial.print("Drehzahl pro Minute: ");
  int speed = ((counter)*calc) / wheel;
  // calculate revolutions per minute

  Serial.println(speed);
  counter = 0;
  // reset counter
  Timer1.attachInterrupt(output);
  // restart timer
}

void loop() {
  // no loop needed
}

```

[SIM24]

7.7. Tests

Um sicherzugehen, dass der Sensor zuverlässig funktioniert, wird empfohlen folgende Tests anhand des einfachen Beispiels mit Code durchzuführen:

- Lichtschrankenprüfung: Drehe die Lochscheibe manuell und beobachte ob die Status-LED bei jedem Loch kurz aufblinkt.
- Signaltest: Starte den Beispielcode und überprüfe ob im eingestellten Zeitintervall Drehzahlen im seriellen Monitor ausgegeben werden.
- Stabilitätstest: Führe längere Messungen durch um sicherzustellen, dass der Sensor stabile Werte liefert.

- Reaktion auf Lichtverhältnisse: Teste den Sensor bei verschiedenen Lichtverhältnissen um externe Störungen der Lichtschranke zu erkennen.

8. OLED-Display

In diesem Kapitel wird erklärt wie was ein OLED-Display ist, wie es funktioniert und wie es in diesem Projekt eingebunden wird.

8.1. Allgemeine Beschreibung eines OLED-Displays

Ein OLED-Display ist eine Art von Bildschirm, bei der organische Materialien Licht emittieren, wenn Strom durch sie hindurchfließt. "OLED" steht dabei für "Organic Light Emitting Diode".

Im Gegensatz zu LCD-Displays benötigen OLED-Displays keine Hintergrundbeleuchtung, da jede einzelne Pixelzelle selbst leuchtet. Außerdem sind OLED-Displays oft dünner, leichter, bieten einen besseren Kontrast und einen breiteren Betrachtungswinkel. Durch die schnelle Reaktionszeit von OLED-Displays werden sie vor allem in Smartphones, Fernsehern, Smartwatches oder High-End-Monitoren verbaut.

Nachteile von OLED-Displays liegen darin, dass sie teurer sind als LCD-Displays. Außerdem haben vor allem blaue OLED-Displays oft eine kürzere Lebensdauer und im Allgemeinen besteht die Gefahr des Einbrennens (Burn-in). Das bedeutet, dass es bei statischen Bildern dazu kommen kann, dass Reste des Bildes auf dem Display sichtbar bleiben. [Sil25]

8.2. Spezifische Beschreibung OLED-Displays

Es wird ein 2,42 Zoll OLED-Display von Joy-IT verwendet. Dabei handelt es sich um ein Display mit 128 x 64 Pixeln, welches gelbe Schrift auf schwarzem Hintergrund darstellt. Es besitzt eine SPI- und eine I2C-Schnittstelle, sodass es mit dem verwendeten Arduino Nano 33 BLE Sense kompatibel ist. [SIM21a]

8.3. Anschluss des Sensors mit dem Arduino Nano 33 BLE Sense

In Table 8.1 ist gezeigt wie das OLED-Display korrekt an den Arduino Nano 33 BLE Sense angeschlossen wird:

OLED-Display	Arudino Nano 33 BLE Sense
1	GND
2	3V3
4	D9
5	D8
7	A5
8	A4

Table 8.1.: Pinbelegung für I2C-Verbindung zwischen OLED-Display und Arduino Nano 33 BLE Sense [SIM21b]

8.4. Spezifikationen

In Table 8.2 sind technische Daten des OLED-Displays aufgelistet.

Spezifikation	Wert
Typ	Negativ OLED
Auflösung	128 x 64 Pixel
Abmessungen	71 x 50 x 7 mm
Versorgungsspannung	3 - 5 V
Versorgungsstrom	90 mA
Logik Level	3 V
High Level Eingang	mind. 2,4 V
Low Level Eingang	max. 0,6 V
Helligkeit	100 - 120 CD/cm ²
Kontrast	> 2000:1
Blickwinkel	> 160°
zuläss. Betriebstemperatur	-40 °C bis 85 °C
Lagerungstemperatur	-45 °C bis 90 °C

Table 8.2.: Spezifikationen des OLED-Displays [SIM21a]

8.5. Bibliothek

Für die Verwendung des OLED-Displays mit dem Arduino Nano 33 BLE Sense wird die U8g2 by oliver Bibliothek benötigt. [SIM21b]

8.5.1. Installation

Im Folgenden wird erklärt, wie die U8g2 Bibliothek installiert wird.

1. Öffnen der Arduino IDE
2. Klick auf Sketch, Bibliothek einbinden, Bibliotheken verwalten
3. In der Suchleiste U8g2 eingeben
4. Klick auf Installieren (Version aus Figure 8.1 entnehmen)

8.5.2. Code-Beispiel

Auf Code-Beispiele für die U8g2 Bibliothek kann man über Datei, Beispiele, U8g2 zugreifen.

8.6. Einfaches Beispiel mit Code

Mit dem Code-Beispiel kann getestet werden, ob das Display korrekt angeschlossen ist. Wenn die Verbindung korrekt ist, dann erscheint auf dem Display "Hello World".

```
#include <U8g2lib.h>
#include <Wire.h>

// initialize display
U8G2_SSD1309_128X64_NONAME2_1_HW_I2C
u8g2(U8G2_R0; U8X8_PIN_NONE);
```

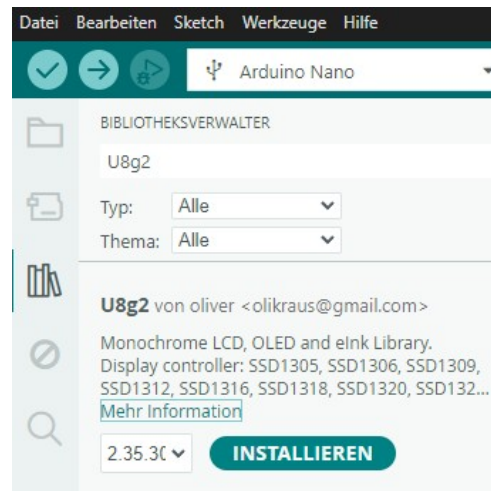


Figure 8.1.: Installation der U8g2 Bibliothek

```
void setup() {  
  u8g2.begin();  
}  
  
void loop() {  
  u8g2.clearBuffer(); // clear image buffer  
  u8g2.setFont(u8g2_font_ncenB08_tr); // choose font  
  u8g2.drawStr(0, 24, "Hello World!"); // write text to buffer  
  u8g2.sendBuffer(); // send buffer to display  
  delay(1000); // wait 1 second  
}
```

[SIM21b]

9. Drehwinkel-Encoder

Der Demonstrator soll mehrere Programme fahren können, deswegen wurde ein Drehwinkel-Encoder der Steuerung hinzugefügt. Dieser wird über drei Pins am Arduino angeschlossen und über zwei weitere Pins wird er mit Spannung versorgt. Durch drehen des Drehschalters werden nacheinander drei Kontakte geschlossen oder geöffnet (ein Kontakt ist immer geschlossen). Dieser dadurch entstehende Signalfuss, bestehend aus zwei um 90 Grad versetzte Sinus bzw. Cosinusschwingungen werden ausgewertet. Daraus wird bestimmt, in welche Richtung (im oder gegen Uhrzeigersinn) und wie weit (inkrementell) gedreht wurde. Mithilfe dieser Logik kann durch ein Menü eine Bewegungsstufe ausgewählt werden, die der Schrittmotor fahren soll.[Bas16] Bei einer Drehung im Uhrzeigersinn wird im Menü eine Bewegungsstufe höher und bei einer Drehung gegen den Uhrzeigersinn eine Bewegungsstufe niedriger ausgewählt. Zusätzlich zum Drehwinkel-Encoder ist auch noch ein Schaltfunktion im Bauteil selbst integriert. Durch eindrücken des Encoders wird ein Taster betätigt, durch den der eingestellte Wert bestätigt und an den Arduino zur weiteren Verarbeitung weitergegeben wird. Zur Besseren Handhabung des Drehwinkel-Encoders wurde noch ein Drehgriff angefertigt und auf dem Drehgeber montiert. Weitere Details:

- **Abmessungen** ($b \times l \times h$): 18 mm $\times$ 31 mm $\times$ 30 mm
- **Betriebsspannung:** 3,3 - 5 V [Sim19]

9.1. Encoder

Die Library Encoder ermöglicht das Zählen von Impulsen aus bestimmten Signalen, die häufig von Drehknöpfen, Motor- oder Wellensensoren sowie anderen Positionsgebern stammen. Diese Bibliothek ist für Arduino und Teensy Boards optimiert und bietet verschiedene Zählmodi, darunter 4X counting für präzise Positionserfassung. Die Verwendung erfolgt durch die Initialisierung eines Encoder-Objekts mit zwei Pins, wobei der erste Pin idealerweise über Interruptfähigkeit verfügt für optimale Leistung. Die Bibliothek unterstützt sowohl I2C- als auch SPI-Schnittstellen und bietet verschiedene Hardware- und Softwareoptionen zur Anpassung an die Anforderungen des Benutzers. Sie wird in diesem Projekt in der Version 1.4.4 verwendet.[Sto24]

9.2. Drehwinkel-Encoder

Um die Funktion und korrekte Ansteuerung des Drehwinkel-Encoders zu testen, wird das Testprogramm 9.1 verwendet. Das Programm wertet die Drehung des Encoders aus und gibt den neuen Zustand im seriellen Monitor der Arduino IDE aus [Sto24].

```
#include <Encoder.h>

const int CLK = 9;
const int DT = 2;
const int SW = 3;
long altePosition = -999;

Encoder meinEncoder(DT, CLK);

void setup()
{
    Serial.begin(9600);
    pinMode(SW, INPUT);
}

void loop()
{
    long neuePosition = meinEncoder.read();

    if (neuePosition != altePosition)
    {
        altePosition = neuePosition;
        Serial.println(neuePosition);
    }

    int StatusTaster = digitalRead(SW);
    if (StatusTaster == 0) {
        Serial.println("Switch_betaetigt");
        Serial.println("Switch_betaetigte");
    }
}
```

Listing 9.1.: Testprogramm für den Encoder

Part IV.

Beschreibung des Hauptprogramms

10. Beschreibung des Hauptprogramms

Das folgende Programm zeigt die vollständige Logik zur Ansteuerung des Schrittmotors, zur Verarbeitung der Sensor- und Schaltereingaben sowie zur Benutzerführung über das OLED-Display.

```
#include <U8x8lib.h>

#define STEP_PIN 2
#define DIR_PIN 3
#define START_BUTTON_PIN 4
#define STOP_BUTTON_PIN 5
#define END_LEFT_PIN 11
#define END_RIGHT_PIN 7
#define MAIN_SWITCH_PIN 8
#define SPEED_SENSOR_PIN 9
#define ENCODER_CLK A1
#define ENCODER_DT A2
#define ENCODER_SW A3

bool motorRunning = false;
bool directionLeft = false;
bool systemActive = false;
bool lastSwitchState = true;
bool endLeftPressed = false;
bool endRightPressed = false;
bool speedSelected = false;
bool measurementFinished = false;
bool measurementActive = false;
bool rideStarted = false;

volatile unsigned long pulseCount = 0;
unsigned long startTime = 0;
unsigned long endTime = 0;
const float DIST_CM = 36.0;

int speedLevel = 1;
int delayValues[] = {1500, 1000, 900, 800, 600};

U8X8_SSD1309_128X64_NONAME2_HW_I2C u8x8(10, A4, A5);
```

Listing 10.1: Definition der Pins und globalen Variablen

```
void countPulse() {
    if (measurementActive) {
        pulseCount++;
    }
}
```

Listing 10.2: Interrupt-Funktion zur Pulserfassung

```

void setup() {
    pinMode(STEP_PIN, OUTPUT);
    pinMode(DIR_PIN, OUTPUT);
    pinMode(START_BUTTON_PIN, INPUT_PULLUP);
    pinMode(STOP_BUTTON_PIN, INPUT_PULLUP);
    pinMode(END_LEFT_PIN, INPUT_PULLUP);
    pinMode(END_RIGHT_PIN, INPUT_PULLUP);
    pinMode(MAIN_SWITCH_PIN, INPUT_PULLUP);
    pinMode(SPEED_SENSOR_PIN, INPUT_PULLUP);
    pinMode(ENCODER_CLK, INPUT);
    pinMode(ENCODER_DT, INPUT);
    pinMode(ENCODER_SW, INPUT_PULLUP);

    attachInterrupt(digitalPinToInterrupt(SPEED_SENSOR_PIN),
        countPulse, FALLING);

    Serial.begin(9600);
    u8x8.begin();
    u8x8.setFont(u8x8_font_chroma48medium8_r);
    u8x8.clear();
    u8x8.drawString(0, 0, "Starte...");
}

```

Listing 10.3: Setup-Funktion zur Initialisierung

```

void loop() {
    bool switchState = digitalRead(MAIN_SWITCH_PIN) ==
        LOW;

    if (switchState != lastSwitchState) {
        lastSwitchState = switchState;
        if (switchState) {
            systemActive = true;
            motorRunning = false;
            speedSelected = false;
            u8x8.clear();
            u8x8.drawString(0, 1, "SYSTEM_AKTIV");
            u8x8.drawString(0, 3, "Willkommen!");
            u8x8.drawString(0, 5, "Stufe_waehlen");
        } else {
            systemActive = false;
            motorRunning = false;
            measurementActive = false;
            u8x8.clear();
            u8x8.drawString(0, 1, "SYSTEM_AUS");
            u8x8.drawString(0, 3, "Auf_Wiedersehen!");
        }
        delay(100);
    }
}

```

Listing 10.4: Loop-Funktion: Hauptlogik und Zustandsschalter

```

if (!systemActive) return;

```

```

// Select speed level
if (!speedSelected) {

```

```

static int lastClk = digitalRead(ENCODER_CLK);
int clk = digitalRead(ENCODER_CLK);
int dt = digitalRead(ENCODER_DT);

if (clk != lastClk && clk == LOW) {
    if (dt == HIGH && speedLevel < 5) speedLevel++;
    if (dt == LOW && speedLevel > 1) speedLevel--;
    u8x8.clear();
    u8x8.drawString(0, 0, "Stufe_waehlen:");
    char buf[16];
    snprintf(buf, sizeof(buf), "Stufe_%d", speedLevel);
    u8x8.drawString(0, 1, buf);
    delay(150);
}
lastClk = clk;

if (digitalRead(ENCODER_SW) == LOW) {
    u8x8.clear();
    u8x8.drawString(0, 0, "Stufe_gewaehlt:");
    char buf[16];
    snprintf(buf, sizeof(buf), "Stufe_%d", speedLevel);
    u8x8.drawString(0, 1, buf);
    u8x8.drawString(0, 3, "START_DRUECKEN");
    speedSelected = true;
    delay(500);
    while (digitalRead(ENCODER_SW) == LOW);
}
return;
}

```

Listing 10.5: Geschwindigkeitswahl über den Encoder

```

// START button pressed
if (digitalRead(START_BUTTON_PIN) == LOW && !motorRunning) {
    if (!rideStarted && digitalRead(END_LEFT_PIN) == HIGH)
        return;

    motorRunning = true;
    measurementActive = true;

    if (!rideStarted) {
        directionLeft = false;
        setDirection();
        startTime = millis();
        measurementFinished = false;
        rideStarted = true;

        u8x8.clear();
        u8x8.drawString(0, 0, "FAHRT_STARTET");
        u8x8.drawString(0, 1, ">_nach_rechts");
    } else {
        setDirection();
        u8x8.clear();
        u8x8.drawString(0, 0, "FAHRT_FORTGESETZT");
        u8x8.drawString(0, 1, directionLeft ? "<=_Rueckfahrt" :

```

```

        "->Hinweg");
    }

    while (digitalRead(START_BUTTON_PIN) == LOW);
}

```

Listing 10.6: Startlogik und Fahrtrichtungssteuerung

```

// STOP possible at any time
if (digitalRead(STOP_BUTTON_PIN) == LOW && motorRunning) {
    motorRunning = false;
    measurementActive = false;
    u8x8.drawString(0, 5, "STOPP_GEDRUECKT");
    while (digitalRead(STOP_BUTTON_PIN) == LOW);
}

```

Listing 10.7: Stoppllogik während der Fahrt

```

// Right end reached, change direction
bool rightNow = digitalRead(END_RIGHT_PIN) == LOW;
if (rightNow && !endRightPressed && motorRunning && !directionLeft) {
    directionLeft = true;
    setDirection();
    u8x8.clear();
    u8x8.drawString(0, 0, "ENDE_RECHTS");
    u8x8.drawString(0, 1, "<=Rueckfahrt");
    delay(300);
}
endRightPressed = rightNow;

```

Listing 10.8: Rechtsanschlag: Richtungswechsel

```

// Left end reached, end ride
bool leftNow = digitalRead(END_LEFT_PIN) == LOW;
if (leftNow && !endLeftPressed && motorRunning && directionLeft) {
    motorRunning = false;
    measurementActive = false;
    measurementFinished = true;
    rideStarted = false;
    endTime = millis();

    float seconds = (endTime - startTime) / 1000.0;
    float speed = DIST_CM / seconds;

    u8x8.clear();
    u8x8.drawString(0, 0, "FAHRT_BEENDET");
    char buf[16];
    snprintf(buf, sizeof(buf), "%.2f cm/s", speed);
    u8x8.drawString(0, 1, buf);
    delay(500);
}
endLeftPressed = leftNow;

```

Listing 10.9: Linksanschlag: Fahrtende und Anzeige

```

// Motor movement
if (motorRunning) {

```

```

        digitalWrite(STEP_PIN, HIGH);
        delayMicroseconds(delayValues[speedLevel - 1]);
        digitalWrite(STEP_PIN, LOW);
        delayMicroseconds(delayValues[speedLevel - 1]);
    }
}
void setDirection() {
    digitalWrite(DIR_PIN, directionLeft ? LOW : HIGH);
}

```

Listing 10.10: Motorschritte und Richtungsfunktion

Zu Beginn wird die Bibliothek `U8x8lib` eingebunden, um ein OLED-Display anzusteuern. Danach werden alle Pins für den Schrittmotor, die Endschalter, den Hauptschalter, den Start- und Stopptaster, den Geschwindigkeitssensor sowie den Drehencoder definiert. Auch mehrere globale Variablen werden angelegt, um Zustände wie „Motor läuft“, „System aktiv“, „Endschalter gedrückt“ oder „Geschwindigkeit gewählt“ zu speichern. Außerdem gibt es Variablen für die Zeitmessung, die Weglänge von 36 cm, die aktuelle Geschwindigkeitsstufe sowie ein Array mit Delay-Werten, die die Motor-Geschwindigkeit beeinflussen. Das OLED-Display wird als Objekt vorbereitet. Listing 10.1

Es gibt eine Interruptfunktion, die bei jedem fallenden Signal des Geschwindigkeitssensors einen Puls zählt. Diese Pulse werden später genutzt, um die gefahrene Geschwindigkeit zu berechnen. Listing 10.2

Im Setup werden alle Pins entsprechend konfiguriert. Motorpins als Ausgänge, Schalter und Taster als Eingänge, der Geschwindigkeitssensor als Eingang mit aktiviertem Interrupt, und auch der Encoder wird eingerichtet. Das OLED-Display wird initialisiert und zeigt eine Startnachricht. Listing 10.3

Im Loop, also der Hauptschleife, wird zunächst geprüft, ob der Hauptschalter eingeschaltet ist. Je nach Zustand zeigt das OLED-Display „SYSTEM AKTIV“ oder „SYSTEM AUS“. Bei Systemstart wird das Display zurückgesetzt und begrüßt den Benutzer, gleichzeitig werden interne Statusvariablen auf einen definierten Zustand gesetzt. Listing 10.4

Sobald das System aktiv ist, kann die Geschwindigkeit gewählt werden. Dazu liest das Programm die Signale vom Drehencoder. Durch Drehen wählt man zwischen fünf Geschwindigkeitsstufen, die in einem Array als unterschiedliche Delay-Werte für den Motor abgelegt sind. Auf dem OLED-Display wird immer die aktuell ausgewählte Stufe angezeigt. Bestätigt wird die Wahl mit dem Encoder-Taster. Danach zeigt das Display an, dass der Benutzer zum Starten bereit ist. Listing 10.5

Drückt der Benutzer den Startknopf, prüft das Programm zunächst, ob es sich um den ersten Start handelt. Falls ja, muss der Wagen am linken Endschalter stehen, bevor er losfahren darf. Wird gestartet, beginnt die Fahrt nach rechts. Die Fahrtrichtung wird gesetzt, die Zeitmessung gestartet und das Pulszählen aktiviert. Auf dem OLED-Display wird angezeigt, dass die Fahrt beginnt und in welche Richtung der Wagen fährt. Listing 10.6

Während der Fahrt kann jederzeit der Stoppknopf gedrückt werden. Sobald dies passiert, stoppt der Motor sofort, das Pulszählen wird angehalten und das OLED-Display zeigt „STOPP GEDRUECKT“ an. Nach Loslassen des Stopptasters kann später wieder gestartet werden. Listing 10.7

Erreicht der Wagen den rechten Endschalte, erkennt das Programm dies und ändert automatisch die Fahrtrichtung. Der Wagen fährt jetzt zurück nach links. Das OLED-Display zeigt an, dass die Rückfahrt beginnt. Listing 10.8

Wenn der Wagen schließlich wieder den linken Endschalte erreicht, wird die Fahrt beendet. Der Motor wird gestoppt, die Zeitmessung abgeschlossen und die Pulszählung deaktiviert. Das OLED-Display zeigt nun die gemessene Zeit sowie die aus Weg und Zeit berechnete Durchschnittsgeschwindigkeit an. Listing 10.9

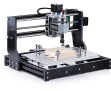
Wenn der Motor aktiv ist, wird im Hauptloop bei jedem Schleifendurchlauf ein einzelner Schritt ausgeführt. Dabei wird der STEP-Pin kurz auf HIGH und danach wieder auf LOW gesetzt, wodurch ein Schrittpuls erzeugt wird. Die Geschwindigkeit des Motors wird über die Pausenzeit zwischen den Impulsen geregelt, die aus dem gewählten Geschwindigkeitslevel abgeleitet wird.

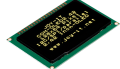
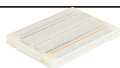
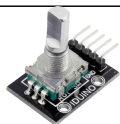
Die Funktion `setDirection()` legt dabei die Drehrichtung des Motors fest. Abhängig von der Richtung links oder rechts wird der DIR-Pin entsprechend gesetzt. So kann der Wagen zuverlässig in beide Richtungen fahren. Listing 10.10

Part V.

Anhang

11. Materialliste

Pos.	Stk.	Bezeichnung	Artikel-Nr.	Preis [€]	Abbildung	Bestelladresse
1	1	Genmitsu 3018 Pro CNC Fräsemaschine	101-60-280PRO	219,00		www.amazon.de
2	1	Arduino Nano 33 BLE Sense Rev.2 nRF52840, mit Header	ARD NANO 33BSH2	47,70		www.reichelt.de
5	1	65 Jumper Wire Kabel im Set	RBS10022	1,55		www.roboter-bausatz.de
4	1	40 Pin Dupont / Jumper Kabel Buchse-Buchse 20 cm	RBS101126	0,98		www.roboter-bausatz.de
5	1	40 Pin Jumper Kabel Buchse-Stecker 20 cm	RBS10028	1,15		www.roboter-bausatz.de
6	1	Arduino - Grove Universal-Kabel, 4-Pin, 20 cm	GRV CA-BLE4PIN20	2,50		www.reichelt.de
7	1	TRU COMPO-NENTS DC-13M Niedervolt-Steckverbinder Stecker, gerade 5.5 mm 2.5 mm 1 St.	1572162 - VQ	3,79		www.conrad.de
8	1	Steckernetzteil, 12 W, 12 V, 1 A	NKSK 200 SW GEW	3,10		www.reichelt.de
9	1	Drucktaster 0,1A-24VAC 1x (Ein) Ø11/9,1mm rt	RAFI 107.301	2,99		www.reichelt.de

10	1	Drucktaster 0,1A-24VAC 1x (Ein) Ø11/9,1mm gn	RAFI 107.507	2,90		www.reichelt.de
11	2	Snap-Action-Mikroschalter, 1x UM, Rollenhebel	AV 32543 AT	1,80		www.reichelt.de
12	1	Entwicklungsboard Schrittmotors- teuerung, A4988	DEBO DRV A4988	5,80		www.reichelt.de
13	1	Entwicklerboards - Display OLED, 2,42", 128x64 Pixel	DEBO OLED 2.42	18,90		www.reichelt.de
14	1	Sunlu PLA Filament 1.75mm 1kg	RBS16624	17,95		www.roboter-bausatz.de
15	1	Experimentier-Slide-Steckboard, 300/100 Kontakte	STECKBOARD S4	4,20		www.reichelt.de
16	1	Kabelbinder-Set, verschiedene Größen, mehrfarbig, 200er-Pack	DELOCK 18628	3,85		www.reichelt.de
17	1	Drehwinkel-Encoder, KY-040	DEBO EN-CODER	2,20		www.reichelt.de
18	1	Wippschalter, 2x Aus, schwarz, I-O	WIPPE 1858.1103	2,60		www.reichelt.de
19	4	Flach-Senkkopfschrauben, Kreuzschlitz, M3, 10 mm	SKS M3X10-50	1,75		www.reichelt.de
20	4	Sechskantmuttern, Edelstahl A2, M3	SK-E M3-100	4,55		www.reichelt.de

21	1	Unterlegscheiben, 10,5 mm	SKU 10,5-100	5,35		www.reichelt. de
22	1	Elektroniklot EL 324 bleifrei 70 g	CFH 52324	10,99		www.reichelt. de
23	1	Entwicklerboards - Speed-Sensor LM393	DEBO SPEED SENS	2,10		www.reichelt. de

Table 11.1.: Materialliste

Literaturverzeichnis

- [Ard24] Arduino Documentation. *Getting Started with Arduino IDE 2*. 2024. URL: <https://docs.arduino.cc/software/ide-v2/tutorials/getting-started-ide-v2/>.
- [Ard25] Arduino. *ABX00031-datasheet*. 2025. URL: <https://docs.arduino.cc/resources/datasheets/ABX00031-datasheet.pdf>.
- [Arm20] Arm. *Arm-Cortex-M4-Processor-Datasheet*. 2020. URL: <https://developer.arm.com/documentation/102832/latest/>.
- [Ava15] Avago Technologies. *Datasheet - APDS-9960 - Digital Proximity, Ambient Light, RGB and Gesture Sensor*. 2015. URL: https://cdn.sparkfun.com/assets/learn_tutorials/3/2/1/Avago-APDS-9960-datasheet.pdf.
- [Bab23] G. Babel. *Elektrische Antriebe in der Fahrzeugtechnik: Lehr- und Arbeitsbuch*. 5. Auflage. Wiesbaden and Heidelberg: Springer Vieweg, 2023. ISBN: 978-3-658-40585-4. DOI: 10.1007/978-3-658-40586-1.
- [Bas16] S. Basler. *Encoder und Motor-Feedback-Systeme*. Wiesbaden: Springer Fachmedien Wiesbaden, 2016. ISBN: 978-3-658-12843-2. DOI: 10.1007/978-3-658-12844-9. URL: <https://link.springer.com/book/10.1007/978-3-658-12844-9>.
- [Ber18] H. Bernstein. *Elektrotechnik/Elektronik für Maschinenbauer: Einfach und praxisgerecht*. 3., überarbeitete Auflage. Lehrbuch. Wiesbaden and Heidelberg: Springer Vieweg, 2018. ISBN: 978-3-658-20837-0. DOI: 10.1007/978-3-658-20838-7.
- [Ber20] H. Bernstein. *Mikrocontroller: Grundlagen der Hard- und Software der Mikrocontroller ATtiny2313, ATtiny26 und ATmega32*. 2., aktualisierte und erweiterte Auflage. Lehrbuch. Wiesbaden and Heidelberg: Springer Vieweg, 2020. ISBN: 978-3-658-30067-8.
- [Fau20] Faulhaber Drive Systems. *FAULHABER Tutorial: Schrittverluste verhindern bei Schrittmotoren*. Ed. by DR. FRITZ FAULHABER GMBH & CO. KG. Schönaich · Deutschland, 2020. URL: <https://www.faulhaber.com/de/know-how/tutorials/schrittmotoren-tutorial-schrittverluste-verhindern-bei-schrittmotoren/>.
- [GW22] W. Gehrke and M. Winzker. *Digitaltechnik: Grundlagen, VHDL, FPGAs, Mikrocontroller*. 8. Auflage. Berlin and Heidelberg: Springer Vieweg, 2022. ISBN: 9783662639535. URL: <https://link.springer.com/book/10.1007/978-3-662-63954-2>.
- [Hag21] R. Hagl. *Elektrische Antriebstechnik*. 3., überarbeitete und erweiterte Auflage. München: Hanser, 2021. ISBN: 978-3-446-46572-5. DOI: 10.3139/9783446468214.
- [JOY19] JOY-IT. *NEMA17-04 - Zweipoliger Schrittmotor*. 2019. URL: https://cdn-reichert.de/documents/datenblatt/A300/NEMA17-04_DN_DE.pdf.

- [MS21] A. Meroth and P. Sora. *Sensornetzwerke in Theorie und Praxis*. Wiesbaden: Springer Fachmedien Wiesbaden, 2021. ISBN: 978-3-658-31708-9. DOI: 10.1007/978-3-658-31709-6. URL: <https://link.springer.com/book/10.1007/978-3-658-31709-6>.
- [Mar24] Marc McComb. *Micro-stepping for Stepper Motors*. Ed. by Microchip Technology. 2024.
- [Nor23] Nordic Semiconductor. *nRF5340 Product Specification: QSPI — Quad serial peripheral interface*. 2023. URL: <https://infocenter.nordicsemi.com>.
- [Nor24a] Nordic Semiconductor. *nRF52840 Product Specification 2*. 2024. URL: <https://infocenter.nordicsemi.com>.
- [Nor24b] Nordic Semiconductor. *nRF52840 Product Specification Memory*. 2024. URL: <https://infocenter.nordicsemi.com>.
- [Nor24c] Nordic Semiconductor. *nRF9161 Product Specification: Cryptocell-ARM TrustZone CryptoCell 310*. 2024. URL: <https://infocenter.nordicsemi.com>.
- [PA22] Petr Filipi and Arduino Tech Support Team. *Difference_between_A33BLESense_and_SenseLite: A Difference between A N 33 BLE Sense vs. Sense Lite*. 2022. URL: <https://forum.arduino.cc/t/a-difference-between-a-n-33-ble-sense-vs-sense-lite/1030305>.
- [Par20] R. Parthier. *Messtechnik - Vom SI-Einheitensystem über Bewertung von Messergebnissen zu Anwendungen der elektrischen Messtechnik*. Mittweida, Deutschland: Springer Vieweg Wiesbaden, 2020. ISBN: 978-3-658-27131-2. DOI: 10.1007/978-3-658-27131-2.
- [SIM20] SIMAC Electronics GmbH. *SPEEDSENSOR SEN-Speed*. 2020. URL: <https://cdn-reichelt.de/documents/datenblatt/A300/DATASHEET-SEN-SPEED.pdf>.
- [SIM21a] SIMAC Electronics GmbH. *2,42 ZOLL OLED DISPLAY COM-OLED 2.42*. 2021. URL: https://cdn-reichelt.de/documents/datenblatt/A300/COM-OLED2.42_DATENBLATT_2021-03-11.pdf.
- [SIM21b] SIMAC Electronics GmbH. *OLED-DISPLAYMODUL COM-OLED2.42*. 2021. URL: https://cdn-reichelt.de/documents/datenblatt/A300/COM-OLED2.42_ANLEITUNG_2021-02-05.pdf.
- [SIM24] SIMAC Electronics GmbH. *GESCHWINDIGKEITSSENSOR Speedsensor LM393 mit Lochscheibe*. 2024. URL: <https://cdn-reichelt.de/documents/datenblatt/A300/SEN-SPEED-ANLEITUNG-20201015.pdf>.
- [SK21] D. Schröder and R. Kennel. *Elektrische Antriebe – Grundlagen*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2021. ISBN: 978-3-662-63100-3. DOI: 10.1007/978-3-662-63101-0.
- [STM15] STMICROELECTRONICS. *Datasheet - LSM9DS1- iNEMO inertial module: 3D accelerometer, 3D gyroscope, 3D magnetometer*. Ed. by STMICROELECTRONICS. 2015. URL: <https://www.st.com/en/mems-and-sensors/lsm9ds1.html>.
- [STM17] STMICROELECTRONICS. *Datasheet - LPS22HB-MEMS nano pressure sensor: 260-1260 hPa absolute digital output barometer*. Ed. by STMICROELECTRONICS. 2017. URL: <https://www.st.com/en/mems-and-sensors/lps22hb.html>.
- [STM21] STMICROELECTRONICS. *Datasheet - MP34DT05-A - MEMS audio sensor omnidirectional digital microphone*. Ed. by STMICROELECTRONICS. 2021.

- [Sil25] Silvio Göthel. *OLED Prinzip und Verwendung, im Besonderen in der Displaytechnik*. 2025. URL: https://tu-dresden.de/ing/informatik/ti/vlsi/ressourcen/dateien/dateien_studium/dateien_lehstuhlseminar/vortraege_lehrstuhlseminar/folder-2010-04-30-7782863372/OLED_Vortrag.pdf.
- [Sim19] Simac Electronics GmbH. *COM-KY040RE-Datenblatt: Drehencoder mit Tasterfunktion*. [\[pdf\]](#). 2019. URL: www.joy-it.net.
- [Sto24] P. Stoffregen. *Encoder Library*. Ed. by PJRC. [\[pdf\]](#). 2024. URL: https://www.pjrc.com/teensy/td_libs_Encoder.html.
- [Tex11] Texas Instruments Incorporated. *STEPPER MOTOR CONTROLLER IC*. 2011. URL: https://cdn-reichelt.de/documents/datenblatt/A300/SOLDERED_DRV8825_DATASHEET.pdf.
- [Tex14] Texas Instruments Incorporated. *DRV8825 Stepper Motor Controller IC*. 2014. URL: <https://www.ti.com/de/lit/gpn/drv8825>.
- [Tex25] Texas Instruments Incorporated. *LM393B, LM2903B, LM193, LM293, LM393 and LM2903 Dual Comparators*. 2025. URL: <https://www.ti.com/lit/gpn/lm393>.

Index

- File
 - blink.ino, 27, 28
 - blnik.ino, 23
- Inertial Measurement Unit
 - see* IMU, 11, 32
- AAR, 11, 35
- ADC, 11, 35
- Advanced Encryption Standard
 - see* AES, 11, 35
- AES, 11, 35
- Analog to Digital Converter
 - see* ADC, 11, 35
- Automatic Address Recognition
 - see* AAR, 11, 35
- BLE, 11, 31, 32, 34
- Bluetooth Low Energy
 - see* BLE, 11, 31
- CCM, 11, 35
- Counter with CBC-MAC
 - see* CCM, 11, 35
- Direkt Memory Access
 - see* DMA, 11, 35
- DMA, 11, 35
- I²C, 11, 32–34
- IC, 11, 33
- IMU, 11, 32
- Inter Circuit
 - see* IC, 11, 33
- Inter-Integrated Circuit
 - see* I²C, 11, 32
- LED, 11, 32
- Light Emitting Diode
 - see* LED, 11, 32
- Mini Universal Serial Bus
 - see* MSB, 11, 35
- MUSB, 11, 35
- Near Field Communication
 - see* NFC, 11, 35
- NFC, 11, 35
- OLED, 11, 34
- Organic Light Emitting Diode
 - see* OLED, 11, 34
- PDM, 11, 33
- Pulse Density Modulation
 - see* PDM, 11, 33
- SCL, 11, 34
- SDA, 11, 34
- Serial Clock
 - see* SCL, 11, 34
- Serial Data
 - see* SDA, 11, 34
- Serial Peripheral Interface
 - see* SPI, 11, 33
- SMD, 11, 33
- SoC, 11, 31
- SPI, 11, 33, 35
- Surface-Mounted Device
 - see* SM, 11, 33
- System on Chip
 - see* SoC, 11, 31
- Ultraviolettstrahlung
 - see* UV, 11, 32
- UV, 11, 32